
ML - Machine Learning

Transformers

Riccardo Berta

2026.05.20

Contents

1	Transformers	2
1.1	Input Representation	3
1.1.1	Tokenisation	4
1.1.2	Embeddings	8
1.2	Attention	9
1.2.1	Queries, keys and values	9
1.2.2	Attention pooling	11
1.2.3	Interpretability	13
1.3	Scoring functions	16
1.3.1	Distance-based scoring	16
1.3.2	Learnable scoring	19
1.4	Machine Translation	21
1.4.1	Preprocessing	24
1.4.2	World-level Tokenization	25
1.4.3	Vocabulary	27
1.4.4	Pipeline	30
1.4.5	Word embeddings	32
1.4.6	Evaluation	34
1.5	An Encoder–Decoder solution	37
1.5.1	Encoder	38
1.5.2	Decoder	40
1.5.3	Seq2seq model	41
1.5.4	Loss function	42
1.5.5	Training loop	44
1.5.6	Prediction	50
1.5.7	Performance	53
1.6	Overcoming the bottleneck	55
1.6.1	Adding attention	56
1.6.2	Bahdanau model	59
1.6.3	Training	60
1.6.4	Performance	62
1.6.5	Showing attention	65
1.7	The transformer architecture	69
1.7.1	Self-attention	69
1.7.2	Multi-head attention	71
1.7.3	Positional encoding	73

1.7.4	Encoder block	78
1.7.5	Decoder block	80
1.7.6	The full transformer	83
1.7.7	Performance	86

1 Transformers

The success of deep learning has been driven largely by a small set of core architectures, namely **multilayer perceptrons**, **convolutional networks**, and **recurrent networks**. Despite the impressive progress achieved with these models, their fundamental structure remained closely aligned with earlier formulations. Although a wide range of methodological improvements enriched the practitioner’s toolkit (such as ReLU activations, residual connections, batch normalization, dropout, and adaptive optimization methods) the underlying architectures can still be understood as **scaled-up and refined versions of classical ideas**. In this perspective, the remarkable advances in deep learning are often attributed less to radically new model designs and more to external factors, in particular the **dramatic increase in computational capabilities**, enabled by parallel hardware such as GPUs, and the **availability of large-scale datasets**, which made it possible to effectively train increasingly expressive models.

However, a new architecture, the **Transformer**, has emerged as a dominant paradigm across a wide range of domains. Initially developed for natural language processing tasks such as machine translation, question answering, and text generation, it has since achieved state-of-the-art performance in areas as diverse as computer vision (e.g., image classification, object detection, segmentation) and spatio-temporal modeling (e.g., weather forecasting, traffic prediction, video understanding). The success of the Transformer stems from its **distinct architectural design**, which abandons both recurrence and convolution in favor of a **self-attention mechanism**. This mechanism enables the model to capture long-range dependencies and model complex, context-dependent interactions between input elements in a flexible and scalable way. Perhaps even more significantly, Transformers introduce a **unified modeling framework**. Once data is expressed in a suitable representation, the same architecture can be applied without substantial modification, regardless of whether the input consists of text, images, audio, or multimodal data. This shift reduces the reliance on specialized architectures tailored to specific modalities (such as convolutions for images or recurrent structures for sequences) and instead promotes a view in which all inputs are mapped into a **shared representation space**, where the model itself learns how to structure and exploit their relationships.

1.1 Input Representation

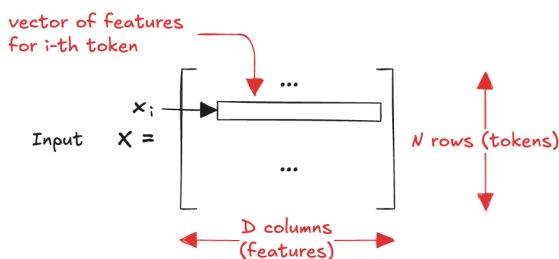
A Transformer does not operate directly on **raw inputs** such as characters, pixels, or audio samples. Instead, its native input consists of a **sequence of tokens**, each represented by a vector of size D :

$$x_i \in \mathbb{R}^D$$

By collecting N such tokens and stacking their vector representations row-wise, we obtain the input matrix:

$$X \in \mathbb{R}^{N \times D}$$

Each column corresponds to one **feature dimension** shared across all tokens



Although this matrix has the same shape as those used in many standard models, its interpretation is fundamentally different. In a typical MLP setting, each row corresponds to an independent data point and each column to an input feature with a fixed, predefined meaning. In a Transformer, instead, **each row represents a token within a single example**, and the columns no longer correspond to explicit features, but to the **coordinates of that token in a shared, learned representation space**. These dimensions do not have an individual semantic interpretation; rather, meaning is encoded collectively across the vector, and is learned from data rather than specified in advance.

The crucial point is that, from the Transformer's perspective, the input is simply the matrix X . Any modality-specific structure (such as word boundaries in text, spatial locality in images, or time-frequency patterns in audio) **must be handled before this stage**. This is achieved by first **tokenizing** the raw signal into discrete units (e.g., words, subwords, image patches, or audio frames), and then **embedding** each unit into a numerical vector. Once the matrix X is constructed, the Transformer operates on it without any explicit knowledge of the original data modality, effectively treating all inputs in a uniform representation space.

1.1.1 Tokenisation

Tokenisation is not a one-size-fits-all procedure: it depends on both the input modality and the task at hand. The underlying goal, however, remains the same: to transform a structured raw input into a collection of vectors of fixed dimension D , so that the Transformer can process them as a sequence of comparable units and apply the same operations to each of them. What varies across domains is therefore **what is chosen as a token** and **how it is mapped into a vector** representation.

In **text**, the most natural tokens are simply units of a sentence. A straightforward approach is to split a sentence into **words**, but in practice it is often more effective to use **subword** tokenization, where words are decomposed into smaller fragments. This allows the model to represent rare words, inflected forms, or proper names without requiring a separate vocabulary entry for every possible variant. After this segmentation step, each token is mapped to an integer index drawn from a **fixed vocabulary**. From this point onward, the model no longer operates on text, but on sequences of discrete identifiers that can be further transformed into vector representations. A simple example helps make this idea concrete. Suppose we start from the following **corpus**, that is, a collection of text data used to train or evaluate a model. A corpus typically consists of many sentences (or documents) drawn from some domain, and serves as the source from which the model learns patterns in language.

```
# Define a small corpus of sentences
corpus = [
    "the cat sat on the mat",
    "the cat ate the rat",
    "the rat sat on the mat",
]
```

We begin by extracting the **vocabulary**, that is, the set of all distinct tokens that appear in the corpus. Each token is then assigned a unique integer identifier, allowing the text to be represented as a sequence of numerical indices that the model can process.

```
# Vocabulary: a collection of all unique words in the corpus
words = []

# Extract words from each sentence and add them to the list
for sentence in corpus:
    words.extend(sentence.split())

# Remove duplicates and sort
```

```
vocab_list = sorted(set(words))

# Assign an index to each word
vocab = {}
for idx, word in enumerate(vocab_list):
    vocab[word] = idx

# Print the vocabulary
print("Vocabulary:", vocab)
```

Vocabulary: {'ate': 0, 'cat': 1, 'mat': 2, 'on': 3, 'rat': 4, 'sat': 5, 'the': 6}

Given a new sentence, we can tokenize it by first splitting it into words and then replacing each word with its corresponding integer identifier from the vocabulary, thereby converting the sentence into a sequence of token IDs that the model can process.

```
# Create an input sentence to be tokenized
sentence = "the cat sat on the mat"

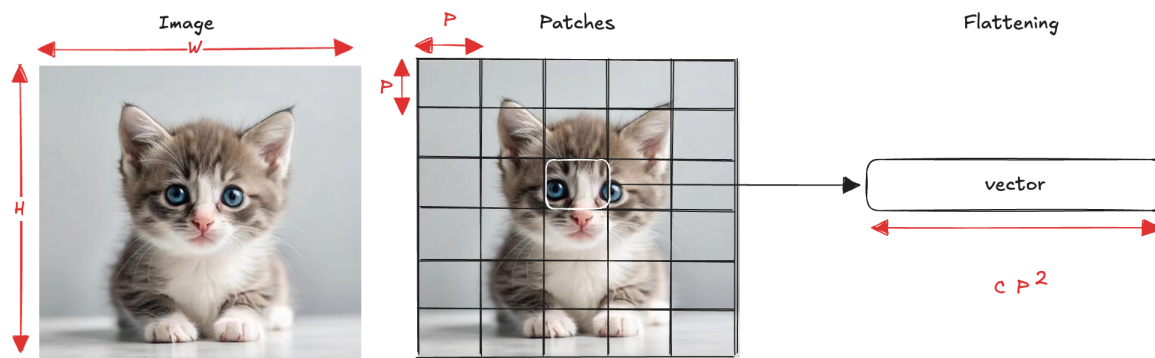
# Split the sentence into words
words = sentence.split()

# Convert each word to its corresponding id using the vocabulary
token_ids = []
for w in words:
    token_ids.append(vocab[w])

# Print the original sentence, the token ids, and the number of tokens
print("Sentence:", sentence)
print("Token IDs:", token_ids)
print("N =", len(token_ids), "tokens")
```

Sentence: the cat sat on the mat
Token IDs: [6, 1, 5, 3, 6, 2]
N = 6 tokens

An **image** can be tokenized by dividing it into non-overlapping patches of size $P \times P$. Each patch contains a number of channels C (e.g., 3 for RGB images), and when the patch is flattened, it is transformed into a vector of length $C \times P \times P$.



As an example, the following function tokenizes a batch of images by splitting each image into non-overlapping patches. The input is assumed to have shape (batch size, number of channels, height, width). Each image is divided into square patches, so the resulting number of patches per image is:

$$N = \left\lfloor \frac{H}{P} \right\rfloor \times \left\lfloor \frac{W}{P} \right\rfloor$$

For each patch, all channel values are flattened into a single vector of length:

$$L = C \times P \times P$$

The output has shape (batch size, number of patches, patch size). This format is usually more convenient than keeping the patch dimension before the sequence dimension, because the model can treat the image as a sequence of N tokens, each with its own feature vector.

```
def image_to_patches(images, P):

    # Extract information: batch, channels, height, width
    B, C, H, W = images.shape

    # Reshape the images in the batch into a grid of patches
    patches = images.reshape(B, C, H//P, P, W//P, P)

    # Reorder dimensions so patches become a sequence
    patches = patches.permute(0, 2, 4, 1, 3, 5)

    # Flatten each patch into a vector
    patches = patches.reshape(B, -1, C*P*P)

    # Return the patches
    return patches
```

In the following example, we construct a dummy batch consisting of two RGB images:

```
import torch

# Create a batch of 2 RGB images of size 224×224
images = torch.randn(2, 3, 224, 224)

# Define the patch size
P = 16

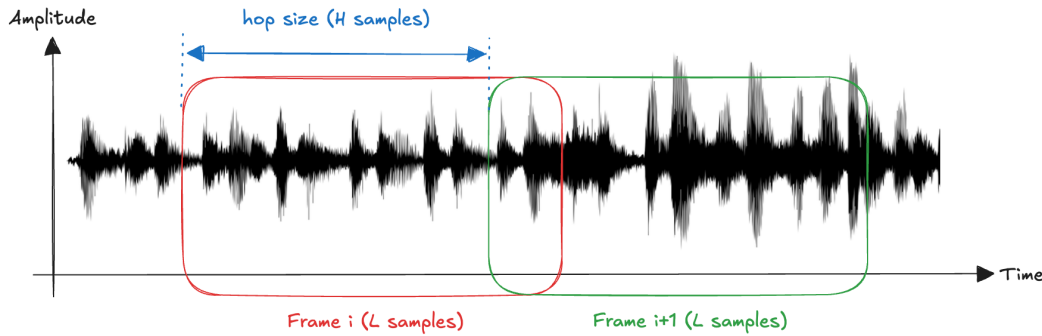
# Convert images into patches
patches = image_to_patches(images, P)

# Print the shapes of the input images and the resulting patches
print("Input shape:", images.shape)
print("Patches shape:", patches.shape)
```

```
Input shape: torch.Size([2, 3, 224, 224])
```

```
Patches shape: torch.Size([2, 196, 768])
```

Unlike text, which is naturally divided into words or subwords, or images, which can be partitioned into spatial patches, an **audio waveform** is a continuous temporal signal with **no intrinsic token boundaries**. Tokenizing audio therefore requires introducing an artificial segmentation in time. The signal is divided into short temporal regions that must be sufficiently compact to keep the sequence length computationally manageable, while still long enough to capture meaningful acoustic structures such as pitch, formants, harmonics, or transient events. The most common strategy is known as **framing**. The waveform is segmented into frames of fixed duration samples, typically corresponding to 20–40 ms of audio. Consecutive frames are generated by sliding a window along the signal with a fixed step size, called the **hop size**. When the hop size equals the frame length, the frames are adjacent and non-overlapping. More commonly, the hop size is smaller than the frame length, so consecutive frames partially overlap. This means that neighboring segments share part of the same waveform, producing a smoother and more redundant representation that preserves temporal continuity and reduces abrupt changes between frames.



Concretely, given an audio signal

$$x[t]$$

the i -th frame is defined as:

$$f_i = x[iH : iH + L] \quad i = 0, 1, \dots, N - 1$$

where N is the total number of frames, which depends on the total number of samples in the signal (T) and the choice of the frame length (L) and hop size (H):

$$N = \left\lfloor \frac{T - L}{H} \right\rfloor + 1$$

A widely used approach is to **tokenize a time–frequency representation** of the signal instead of the raw waveform. In practice, we first apply a Fourier transform to obtain a **spectrogram**, which represents the audio as a 2D array that can then be tokenized similarly to an image. The key intuition is that many perceptually relevant patterns (such as pitch, formants, harmonics, and transient onsets) become much more explicit in the spectrogram domain than in raw temporal samples. Consequently, the resulting tokens tend to encode more meaningful acoustic structure and higher-level semantic information.

1.1.2 Embeddings

Tokenization provides a collection of discrete symbolic units, but these units are not yet suitable for numerical computation. The role of embeddings is to **map each token into a vector** in a **shared D -dimensional space**, transforming heterogeneous inputs into a common numerical representation on which the same learning algorithms can operate. The key idea is that, once embedded, tokens originating from different modalities (or carrying different semantic meanings) become **comparable geometric objects**. Relationships such as similarity, context, or semantic structure can then be expressed through distances and directions in the embedding space. At a high level, an embedding can be viewed simply as a learned function:

$$e : \text{token} \rightarrow \mathbb{R}^D$$

Applying this mapping to the tokens produced during tokenization yields the input representation matrix:

$$X = \begin{bmatrix} e(t_1)^\top \\ e(t_2)^\top \\ \vdots \\ e(t_N)^\top \end{bmatrix} \in \mathbb{R}^{N \times D}$$

The key conceptual point is that embeddings do not merely encode tokens, they **define the geometry** in which it is possible **to measure compatibility**. When two tokens end up with similar embeddings, the model can easily make them attend to similar information, when embeddings are far apart, the model can keep them distinct. We will see several embedding strategies in the examples, but the most common one is to simply learn a **lookup table** that maps each token to a vector. This is the standard approach for text, where we have a fixed vocabulary of tokens and we can learn an embedding matrix. For images and audio, we typically use a **learned linear projection** that maps raw pixel values or spectrogram values to the embedding space.

1.2 Attention

Up to this point, every architecture we have discussed relied (explicitly or implicitly) on the assumption that the **input lives in a fixed, well-defined shape**. In ImageNet classification, for example, we typically resize or crop images to 224x224 pixels, and the network is engineered (and trained) around that convention. In time-series forecasting, the situation is similar: an RNN is fed windows of a predetermined length, so “how much past” the model sees is fixed in advance. This design choice is convenient, but it becomes **a real bottleneck when the input is genuinely variable in size and in information content**. When sequences get long, the difficulty is not merely computational, it is **conceptual**: the model must decide what to retain, what to forget, and how to access something it saw (or produced) many steps ago. In practice, keeping track of “everything so far” becomes increasingly fragile as the context grows.

1.2.1 Queries, keys and values

What we need is a mechanism that can selectively read from a large collection of information, one where the amount of stored data can grow arbitrarily, yet the way we access it remains stable and efficient. A natural place to look for intuition is the **abstraction of a database**: a collection of **items**, each addressed by a **key** and carrying a **value**:

$$D = \{(k_1, v_1), (k_2, v_2), \dots, (k_m, v_m)\}$$

For instance, we can consider a database consisting of the last names and first names of a group of people, with the last name as the key and the first name as the value:

```
# Create a dictionary mapping last names to first names
D = {"Zhang": "Aston",
     "Lipton": "Zachary",
     "Li": "Mu",
     "Smola": "Alex",
     "Hu": "Rachel",
     "Werness": "Brent"}
```

We can then **operate on the database** by issuing a **query**. For example, an exact query to retrieve a specific entry:

```
# Exact match query
q = "Li"

# Execute the query, in that case is a simple dictionary lookup
exact_result = D[q]

# Print the result of the query
print("Exact match: ", exact_result)
```

Exact match: Mu

Or an approximate query to retrieve all entries whose key start with a given prefix:

```
# Approximate match (keys starting with q)
q = "L"

# Execute the query, in that case is a dictionary comprehension
# to filter keys starting with q
approx_results = {k: v for k, v in D.items() if k.startswith(q)}

# Print the results of the approximate match query
print("Approximate matches: ", approx_results)
```

Approximate matches: {'Lipton': 'Zachary', 'Li': 'Mu'}

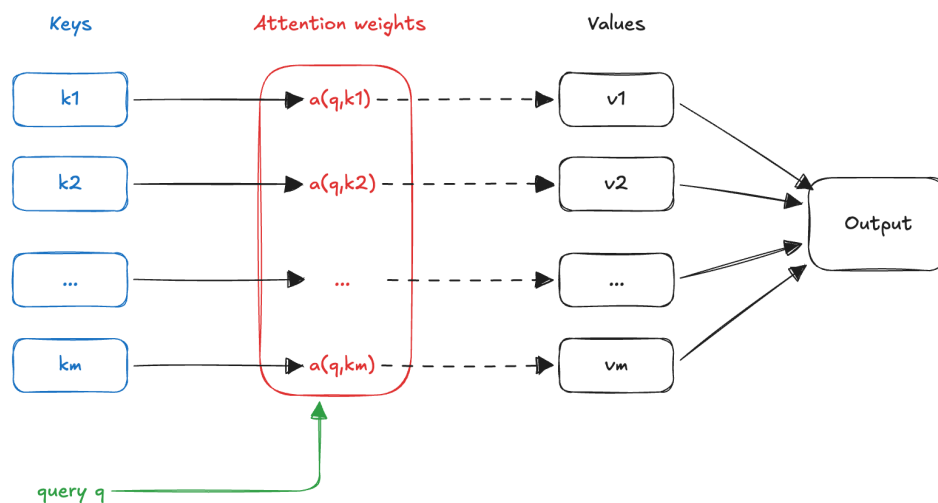
We can imagine more complex queries, such as retrieving the top-k entries most similar to a given text, or retrieving all entries that satisfy some logical condition. The crucial point is that **the size of the database can vary freely** (we can add or remove entries), however the **query mechanism is stable** (we do not need to change the way we query it).

1.2.2 Attention pooling

Even though the example is intentionally simple, it surfaces several important ideas. The first is that we can formulate a query as an operation on key–value pairs in a way that does **not depend on the database size**: the same mathematical object still makes sense whether the database contains a handful of items or millions. The second is that the very **same query can legitimately produce different outputs**, because the answer is not “hard-coded” in the procedure but is induced by the current contents of the database. The third is that operating over an enormous state space **does not require complicated code**: the mechanism can be as elementary as exact lookup, approximate matching, or retrieving the top-k most relevant entries, with the **complexity pushed into the representation and the similarity score** rather than into bespoke control logic. Finally, and perhaps most importantly, none of this requires **compressing or simplifying the database** just to make retrieval workable: we can keep a large collection of stored information and still read from it effectively by learning how to address it. This perspective leads directly to one of the most influential architectural ideas of the last decade: the **attention mechanism**, which can be defined as:

$$A(q, D) = \sum_{i=1}^m a(q, k_i) v_i$$

Here the terms $a()$ are **scalar attention weights** that quantify how strongly the query matches each key. The term attention is natural: the mechanism assigns large weights only to a small subset of entries, and the output is therefore dominated by the corresponding values, those are the items the model is effectively “paying attention” to. Operationally, attention reads from the database by producing a weighted linear combination of the stored values:



The operation of computing a weighted sum of values guided by how well the query matches each key is called **attention pooling**. Like other pooling operations, it aggregates information, but here the aggregation is **adaptive**, shaped by the query rather than fixed in advance. The earlier exact-lookup example is recovered as a limiting case: if one key matches perfectly, its weight becomes 1 while all others go to 0, and the sum collapses to the single retrieved value. Several useful special cases follow from different choices of weights and similarity scores.

A natural next step is to **constrain** the weights so that they truly represent an allocation of attention over the database entries. Formally, we impose the **nonnegativity constraint**:

$$a(q, k_i) \geq 0 \quad \text{for all } i$$

thereby ensuring that attention weights can only enhance or suppress contributions through selection, rather than through cancellation. Without this constraint, the retrieval mechanism could exhibit pathological behavior by effectively "anti-attending" to certain entries, subtracting their associated values from the final result. From the perspective of memory retrieval, such negative coefficients are difficult to interpret. When accessing a memory, we may choose to ignore irrelevant entries, but we do not retrieve the negation of their contents. Nonnegative attention weights therefore enforce a more coherent interpretation of attention as a distribution of relevance over stored information.

We additionally impose the **sum-to-one constraint**:

$$\sum_{i=1}^m a(q, k_i) = 1$$

so that the attention weights acquire the interpretation of a probability distribution over the database entries. Under this constraint, the mechanism must decide **how to distribute a fixed budget of attention** across all keys. This normalization is important both conceptually and computationally. Conceptually, it reinforces the interpretation of attention as selective allocation of

relevance. Computationally, it prevents the mechanism from arbitrarily increasing the magnitude of the output simply by scaling all weights simultaneously; the overall gain remains controlled because increasing attention on one entry necessarily reduces it elsewhere. A standard way to enforce both nonnegativity and normalization is through the softmax function. Starting from an unnormalized compatibility score s , which measures the similarity between the query and the i -th key, attention weights are defined as

$$a(q, k_i) = \frac{\exp(s(q, k_i))}{\sum_{j=1}^m \exp(s(q, k_j))}$$

The exponential guarantees positivity, while the normalization term ensures that all weights sum to one. As a result, entries with higher compatibility scores receive a larger fraction of the available attention mass.

1.2.3 Interpretability

One of the reasons attention mechanisms are so compelling is that they often appear **naturally interpretable**. Because the attention weights are constrained to be nonnegative and to sum to one, they can be viewed as a distribution describing **how strongly the model attends to each key**. In this interpretation, large weights indicate entries that are highly relevant to the current query, whereas small weights correspond to entries that contribute little to the final retrieval. This perspective is particularly appealing because it frames attention as **a soft, content-based retrieval process**: rather than applying an opaque transformation, the model appears to “look up” the information that matters most for the current computation. At the same time, it is important to maintain the correct interpretation of these weights. Attention scores are not explanations explicitly generated by the network, they are simply intermediate computational quantities that happen to admit an intuitive semantic reading. In some cases they correlate strongly with what humans would regard as relevance, but this alignment is neither guaranteed nor necessarily stable across architectures or tasks. Nevertheless, the explicit structure of attention provides **a valuable analytical window into the model’s behavior**. By visualizing how the attention distribution changes as the query varies, we can often gain substantial insight into the retrieval patterns, dependencies, and internal organization learned by the network. To make these attention patterns easier to inspect, we define a `show_heatmap` function that visualizes the attention weights as a **heatmap**, placing queries along the vertical axis and keys along the horizontal axis. In this representation, each cell encodes how strongly a given query attends to a particular key, allowing us to immediately **identify concentration patterns, long-range dependencies, and regions of focused or diffuse attention**.

```
import matplotlib.pyplot as plt

def show_heatmap(matrix, xlabel, ylabel, xticks=None, yticks=None):

    # Create a figure
    plt.figure(figsize=(5, 5))

    # Display the matrix as a heatmap
    pcm = plt.imshow(matrix.detach().numpy(), cmap='Blues', vmin=0, vmax=1)

    # Set labels for axes
    plt.xlabel(xlabel)
    plt.ylabel(ylabel)

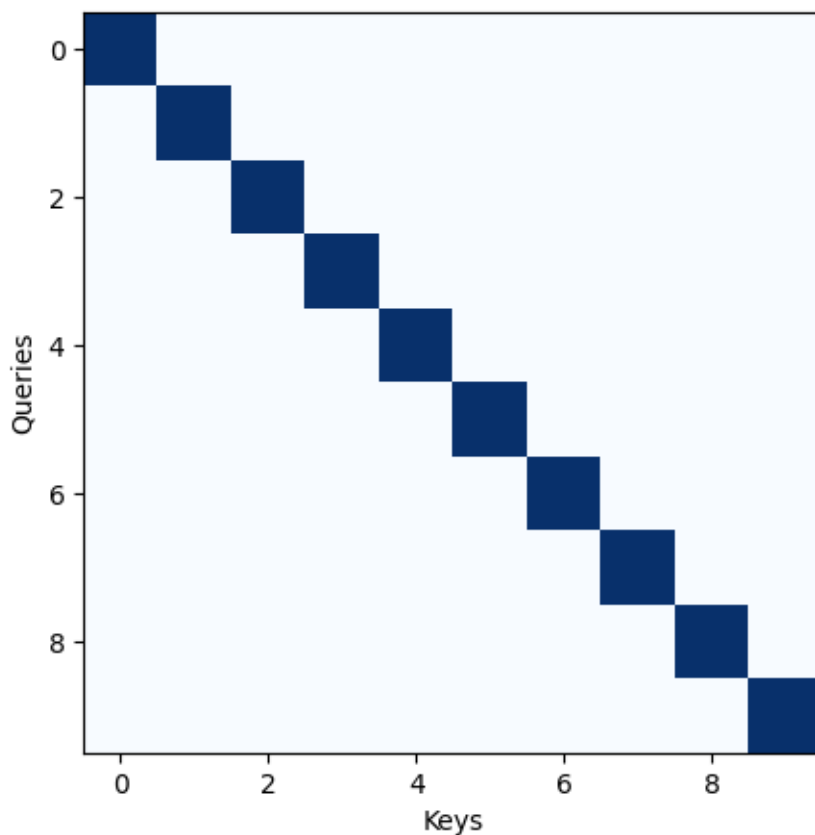
    # Set tick labels if provided
    if xticks is not None:
        plt.xticks(range(len(xticks)), xticks, rotation=45, ha='right')
    if yticks is not None:
        plt.yticks(range(len(yticks)), yticks)

    # Show the plot
    plt.show()
```

As a simple sanity check, let us visualize the **identity matrix**, which corresponds to the idealized case in which **each query assigns all its attention to the matching key and zero attention to every other entry**. In this setting, the attention weight is equal to 1 only when the query and key coincide.

```
# Create a dummy attention weights matrix (10x10 identity matrix)
attention_weights = torch.eye(10)

# Plot the attention weights as a heatmap
show_heatmap(attention_weights, xlabel='Keys', ylabel='Queries')
```

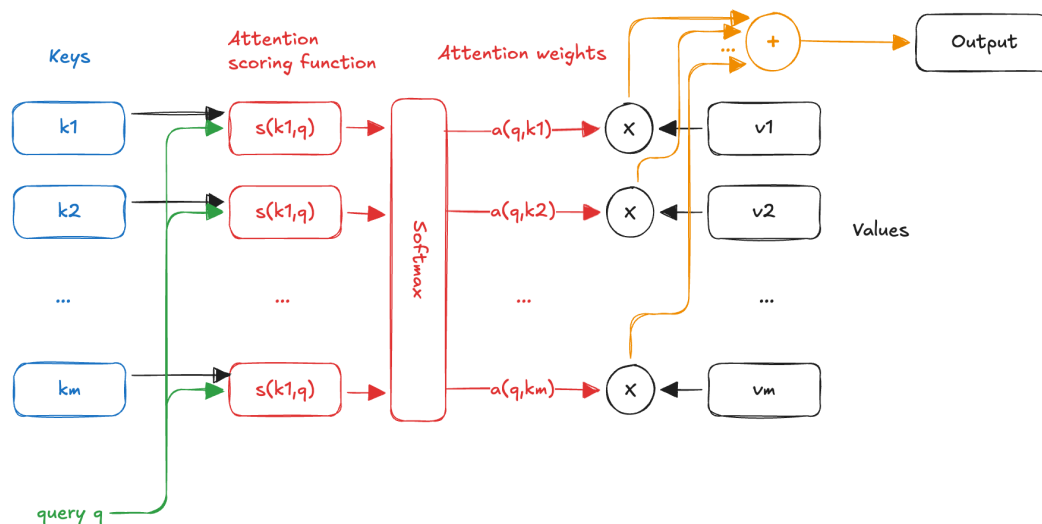


What makes the attention mechanism particularly remarkable is the contrast between the **simplicity of its computation** and **the scale of the space over which it operates**. The underlying procedure (the query together with its similarity function) can remain extremely compact even when the number of keys and values becomes enormous. From the perspective of neural network design, this is a highly desirable property. The mechanism does not require a separate set of learned parameters for every possible memory entry, instead, it reuses the same learned matching rule to dynamically access a potentially vast database. As the number of entries grows, the model must indeed compute more similarity scores and more attention weights, so the runtime cost in computation and memory increases accordingly. However, this increase is computational rather than parametric: the model does not acquire additional degrees of freedom simply because the database becomes larger. The attention weights are generated on the fly by a fixed compatibility function, rather than being stored as independently learned parameters. Equally important, the mechanism itself does not need to be redesigned as the database scales. Attention applies exactly the same retrieval and aggregation procedure whether the memory contains tens, thousands, or millions of entries. **Increasing the database size merely introduces more candidates against which the query must be compared**, it does not alter the structure of the computation. In this sense, attention provides a **scalable and size-agnostic framework** for

content-based information retrieval and aggregation.

1.3 Scoring functions

An attention layer is essentially determined once we specify **how to measure the compatibility between a query and a key**. Everything else follows a standard and reusable computational pipeline. For each key, the mechanism first computes a compatibility score that quantifies how relevant that entry is to the current query. These raw scores are then normalized through a softmax operation, producing attention weights that are nonnegative and sum to one. The resulting weights can therefore be interpreted as a distribution of relevance over the available entries. Finally, the layer retrieves information from the database by forming a weighted combination of the corresponding values, so that entries receiving higher attention contribute more strongly to the output representation:



This is precisely the purpose of the **scoring function**:

$$s(q, k)$$

which takes a query and a key and returns a single scalar measuring their compatibility.

1.3.1 Distance-based scoring

A conceptually simple way to build such a score is to start from **distance**:

$$d(q, k) = \|q - k\|^2$$

and convert it into a similarity score by negating (and often scaling) it:

$$s(q, k) = -\|q - k\|^2$$

This directly encodes the intuitive principle that “closer means more relevant”: keys located near the query in representation space receive larger scores and therefore larger attention weights. Although this formulation is easy to interpret geometrically, it is **not the most computationally efficient choice**. Computing distances and norms is generally less hardware-friendly than the primitive operation modern accelerators are optimized for: matrix multiplication. For this reason, most attention mechanisms in deep learning rely on scoring functions based on dot products, which can be evaluated extremely efficiently and integrate naturally into highly optimized linear-algebra pipelines.

The distance-based similarity score can be rewritten in a form that makes its connection to dot products explicit:

$$s(q, k) = -\|q - k\|^2 = -\|q\|^2 - \|k\|^2 + 2q^\top k$$

The first term depends only on the query and is therefore identical for every key. Adding or subtracting the same constant from all scores does not change the resulting distribution. Consequently, this term can be removed without affecting the attention mechanism:

$$s(q, k) = -\|k\|^2 + 2q^\top k$$

In many deep architectures, activations are intentionally kept within a controlled range through techniques such as batch normalization or layer normalization. Under these conditions, the norms of the keys often vary relatively little, making this term like a small offset which is commonly discarded:

$$s(q, k) = 2q^\top k$$

The multiplicative factor of 2 is itself irrelevant, since uniformly rescaling all scores only changes the temperature of the softmax. This leads to the standard dot-product formulation:

$$s(q, k) = q^\top k$$

One final issue concerns numerical stability. Suppose:

$$q \in \mathbb{R}^d, \quad k \in \mathbb{R}^d$$

If the components of q and k are modeled as independent random variables with zero mean and unit variance, then the dot product has variance proportional to d . As the dimensionality increases, the scores therefore become increasingly large in magnitude. Feeding such values directly into a softmax tends to produce extremely sharp distributions in which one key dominates almost completely, while the remaining weights collapse toward zero. To keep the scale of the scores roughly independent of dimension, we use **scaled dot-product score** dividing by square root of the dimensionality:

$$s(q, k) = \frac{q^\top k}{\sqrt{d}}$$

We can implement a batch version of this function as follows using PyTorch:

```
import torch.nn.functional as F

def scaled_dot_product_attention(queries, keys, values):

    # Find the dimension of the queries/keys
    d = queries.shape[-1]

    # Compute the attention scores
    scores = queries @ keys.transpose(-2, -1) / (d ** 0.5)

    # Normalize the scores to obtain attention weights
    weights = F.softmax(scores, dim=-1)

    # return the weights and the weighted sum of the values
    return weights, weights @ values
```

We have assumed that queries and keys belong to the same vector space, meaning that they share the same dimensionality. While convenient, this assumption is not fundamental to the attention mechanism itself. In many practical settings, queries and keys naturally arise from different representations and may therefore have incompatible dimensions. The obstacle is that the dot product is no longer directly defined when the vectors have different dimensions. A standard solution is to learn two linear projections that map queries and keys into a **common latent space** of dimension h . Compatibility is then computed in this shared representation space rather than in the original coordinate systems. Formally, we introduce two learnable matrices:

$$W_q \in \mathbb{R}^{h \times d_q} \quad W_k \in \mathbb{R}^{h \times d_k}$$

and define the projected representations:

$$\tilde{q} = W_q q \quad \tilde{k} = W_k k$$

The attention score is then computed as:

$$s(q, k) = \frac{\tilde{q}^\top \tilde{k}}{\sqrt{h}} = \frac{(W_q q)^\top (W_k k)}{\sqrt{h}}$$

These learned projections allow the model not only to reconcile dimensional mismatches, but also to discover a representation space in which compatibility relationships become easier to express and optimize. In that case we should design the implementation as a PyTorch class, since we have learnable parameters:

```
import torch.nn as nn

class GeneralizedScaledDotProductAttention(nn.Module):

    def __init__(self, d_q, d_k, h):
        super().__init__()

        # Learnable linear projections for queries
        self.W_q = nn.Linear(d_q, h, bias=False)

        # Learnable linear projections for keys
        self.W_k = nn.Linear(d_k, h, bias=False)

        # Dimensionality of the projected space
        self.h = h

    def forward(self, queries, keys, values):

        # Project queries and keys into a common space of dimension h
        q = self.W_q(queries)
        k = self.W_k(keys)

        output, self.weights = scaled_dot_product_attention(q, k, values)

        # Return the scaled dot-product
        return output
```

1.3.2 Learnable scoring

The scaled dot-product score is efficient and well-motivated, but **it encodes a specific assumption** about what "compatibility" means: two vectors are similar when they point in the same direction. This works well in many cases, but it is a structural constraint: the score is entirely determined by the geometric alignment of the projected query and key vectors. However, the relevant relationship is not necessarily one of geometric alignment, but of **learned semantic compatibility**. In these situations, we would like the scoring function itself to be a **learnable component**, one that can discover from data what "relevant" means for the task at hand. This is the idea behind the **additive score**. Rather than combining the projected query and key by multiplication, we first sum them and then pass the result through a nonlinearity, letting a small learned function shape their interaction:

$$s(q, k) = w^\top \tanh(W_q q + W_k k + b)$$

where

$$b \in \mathbb{R}^h \quad w \in \mathbb{R}^h$$

This gives the mechanism greater expressive flexibility, since the notion of "match" is no longer restricted to correlations captured by an inner product.

```
class AdditiveAttention(nn.Module):
    def __init__(self, d_q, d_k, h):
        super().__init__()

        # Learnable linear projections for queries
        self.W_q = nn.Linear(d_q, h, bias=False)

        # Learnable linear projections for keys
        self.W_k = nn.Linear(d_k, h, bias=False)

        # Bias term
        self.b = nn.Parameter(torch.zeros(h))

        # Learnable linear projection for the output
        self.w = nn.Linear(h, 1, bias=False)

    def forward(self, queries, keys, values):

        # Project queries and keys into a common space of dimension h
        q = self.W_q(queries).unsqueeze(2)
        k = self.W_k(keys).unsqueeze(1)

        # Compute the attention scores using a non-linear activation
        scores = self.w(torch.tanh(q + k + self.b)).squeeze(-1)

        # Normalize the scores to obtain attention weights
        self.weights = F.softmax(scores, dim=-1)

        # Return the weighted sum of the values
        return self.weights @ values
```

Attention scoring functions provide a flexible mechanism for aggregating information over sequences of variable length. Among them, scaled dot-product attention has become the fundamental building block of modern Transformer architectures due to both its simplicity and its

computational efficiency on parallel hardware. When queries and keys originate from representations with different dimensionalities, this framework remains fully applicable. Both generalized dot-product attention and additive attention address the mismatch by learning projections into a shared latent space before computing compatibility scores. In this way, attention retains the same conceptual structure while accommodating heterogeneous representations. Because attention layers sit at the computational core of contemporary large-scale models, improving their efficiency and stability has become a major area of research and engineering. Recent advances focus on optimized kernels, memory-efficient implementations, mixed-precision computation, and hardware-aware execution strategies. A representative example is the **NVIDIA Transformer Engine library**, which provides highly optimized implementations of Transformer components for modern GPU architectures.

1.4 Machine Translation

The abstract framework developed so far (queries, keys, values, and scoring functions) becomes particularly compelling when applied to problems that genuinely require a **flexible, size-independent mechanism for retrieving and aggregating information**. Machine translation is a canonical example of such a problem. In machine translation, the objective is to **transform a sentence in a source language into a semantically equivalent sentence in a target language**. Both the input and the output are variable-length sequences, and, crucially, there is generally no fixed positional correspondence between their elements. A word appearing early in the source sentence may influence a word generated much later in the target sentence, and different languages may reorder grammatical structures in radically different ways. More broadly, machine translation exemplifies a large class of problems involving **mappings between two unaligned sequences**. The challenge is not simply to process sequential data, but to dynamically determine which parts of one sequence are most relevant to each element of the other. Attention mechanisms are particularly well suited to this setting because they allow the model to perform content-based retrieval over the entire input sequence while generating each output element. As a concrete instance of this task, we work with the **English–Italian dataset** from the **Tatoeba Project**, a crowd-sourced collection of bilingual sentence pairs.

```
import urllib.request
import zipfile
import os

# Define the URL of the dataset
url = "https://bit.ly/english-italian-zip"
```

```
# Create data directory if it does not exist
os.makedirs("./data", exist_ok=True)

# Prepare data if not already existing
if not os.path.exists("./data/eng-ita.zip"):
    print("Downloading dataset ... ")

    # Download the dataset
    u = urllib.request.urlopen(url)
    data = u.read()
    u.close()

    # Save the dataset to a local file
    with open("./data/eng-ita.zip", "wb") as f :
        f.write(data)

    # Extract the dataset
    with zipfile.ZipFile("./data/eng-ita.zip","r") as zip_ref:
        zip_ref.extractall("./data")

else:
    print("Dataset already exists. Skipping download.")
```

Dataset already exists. Skipping download.

Each example in the dataset consists of a tab-separated pair of an English source sequence and its Italian translation, accompanied by a CC-BY attribution field that we discard.

```
import pandas as pd

# Load the English-Italian sentence pairs into a Pandas DataFrame
translation_raw_data = pd.read_csv("./data/ita.txt", sep="\t", header=None,
                                   names=["english", "italian", "attribution"],
                                   encoding="utf-8")

# Drop the attribution column (CC-BY licence metadata, not needed for training)
translation_raw_data = translation_raw_data.drop("attribution", axis=1)

# Drop duplicate pairs
translation_raw_data = translation_raw_data.drop_duplicates()
```

```
# Reset index after deduplication
translation_raw_data = translation_raw_data.reset_index(drop=True)

# Show some statistics about the dataset
translation_raw_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 403477 entries, 0 to 403476
Data columns (total 2 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   english    403477 non-null  object
 1   italian    403477 non-null  object
dtypes: object(2)
memory usage: 6.2+ MB
```

It can be useful to inspect a few sentence pairs from the dataset in order to develop an intuition for the structure and variability of the translation task. Examining concrete examples helps highlight important characteristics of the data, such as differences in sentence length, word order, and grammatical structure between the source and target languages.

```
# Extract some random samples from the dataset to inspect
samples = translation_raw_data.sample(5)

# Print the sampled sentence pairs
for i, row in enumerate(samples.itertuples(index=False), 1):
    print(f"Example {i}")
    print(f"English : {row.english}")
    print(f"Italian : {row.italian}")
    print("-" * 60)
```

Example 1

English : I know that you did your best.

Italian : Lo so che ha fatto del suo meglio.

Example 2

English : Tom left me everything.

Italian : Tom mi lasciò tutto.

Example 3

English : I stayed at home.

Italian : Sono rimasta a casa.

Example 4

English : Do you think that you can handle it?

Italian : Pensi di riuscire a gestirla?

Example 5

English : Tom is very liberal, isn't he?

Italian : Tom è molto liberale, vero?

We fix the seed of the random number generator in order to reproduce the results:

```
import numpy as np;
import random;

# Select a seed for reproducibility of results
seed = 42;

# Set the seed for PyTorch, NumPy, and the built-in random module
torch.manual_seed(seed);
np.random.seed(seed);
random.seed(seed);
```

1.4.1 Preprocessing

After downloading the dataset, we apply a series of preprocessing steps to normalize the raw text. These transformations are designed to reduce unnecessary variability and to make tokenization more consistent. For example, non-breaking spaces are replaced with standard spaces, uppercase characters are converted to lowercase, and punctuation marks are separated from adjacent words by inserting whitespace. This ensures that punctuation is treated as an independent token rather than being merged with surrounding text.

```
import re

def preprocess(text):
```

```
# Replace non-breaking spaces with regular spaces
text = text.replace('\u00a0', ' ')

# Lowercase
text = text.lower()

# Add spaces around punctuation
text = re.sub(r'([\w\s])', r' \1 ', text)

# Normalize spaces by replacing multiple spaces with a single space
# and stripping leading/trailing spaces
text = re.sub(r'\s+', ' ', text).strip()

# Return the preprocessed text
return text
```

We can apply this preprocessing to both columns of the dataset:

```
# Apply the preprocessing function to both English and Italian columns
translation_raw_data["english"] = translation_raw_data["english"].apply(preprocess)
translation_raw_data["italian"] = translation_raw_data["italian"].apply(preprocess)
```

1.4.2 World-level Tokenization

To keep the exposition simple, we adopt a **word-level tokenization strategy**, although modern neural translation systems typically employ more sophisticated approaches, such as subword tokenization. Under this scheme, each token corresponds either to a word or to an individual punctuation symbol. The preprocessing pipeline operates on pairs of aligned sentences. Each sentence is first decomposed into a sequence of tokens, after which a special **end-of-sequence token** (`<eos>`) is appended explicitly to the end of the sequence. The procedure ultimately produces two collections of tokenized sequences, each element in these collections is itself a list of tokens. More specifically, `src[i]` contains the tokenized form of the *i*-th source sentence (English in this example), while `tgt[i]` contains the tokenized representation of the corresponding aligned target sentence (Italian):

```
def tokenize(data):

    # Split each sentence into words and prepend beginning-of-sequence token
```

```
# and append an end-of-sequence token
src = [['<bos>'] + row.split() + ['<eos>'] for row in data["english"]]
tgt = [['<bos>'] + row.split() + ['<eos>'] for row in data["italian"]]

# Return the tokenized source and target sentences
return src, tgt
```

As a concrete example, we apply this preprocessing pipeline to the sentence pairs in the dataset and inspect a randomly selected tokenized example to visualize the resulting sequence representation.

```
# Tokenize the English and Italian sentences
eng_sentences, ita_sentences = tokenize(translation_raw_data)

# Select a random index to inspect a random sentence pair
random_idx = random.randrange(len(eng_sentences))

# Show the tokenized English and Italian sentences at the random index
print(eng_sentences[random_idx])
print(ita_sentences[random_idx])
```

```
['<bos>', 'tom', 'works', 'at', 'the', 'local', 'supermarket', '.', '<eos>']
['<bos>', 'tom', 'lavora', 'nel', 'supermercato', 'locale', '.', '<eos>']
```

To better understand the structure of the dataset, let us visualize the distribution of sequence lengths by plotting a histogram of the number of tokens per sentence. In this relatively simple English–Italian translation corpus, the vast majority of sequences contain fewer than 20 tokens.

```
import matplotlib.pyplot as plt

# Compute sentence lengths in terms of number of tokens
eng_lengths = [len(tokens) for tokens in eng_sentences]
ita_lengths = [len(tokens) for tokens in ita_sentences]

# Prepare bins and counts for the histogram
bins = range(1, 25)
eng_counts = [eng_lengths.count(i) for i in bins]
ita_counts = [ita_lengths.count(i) for i in bins]
```

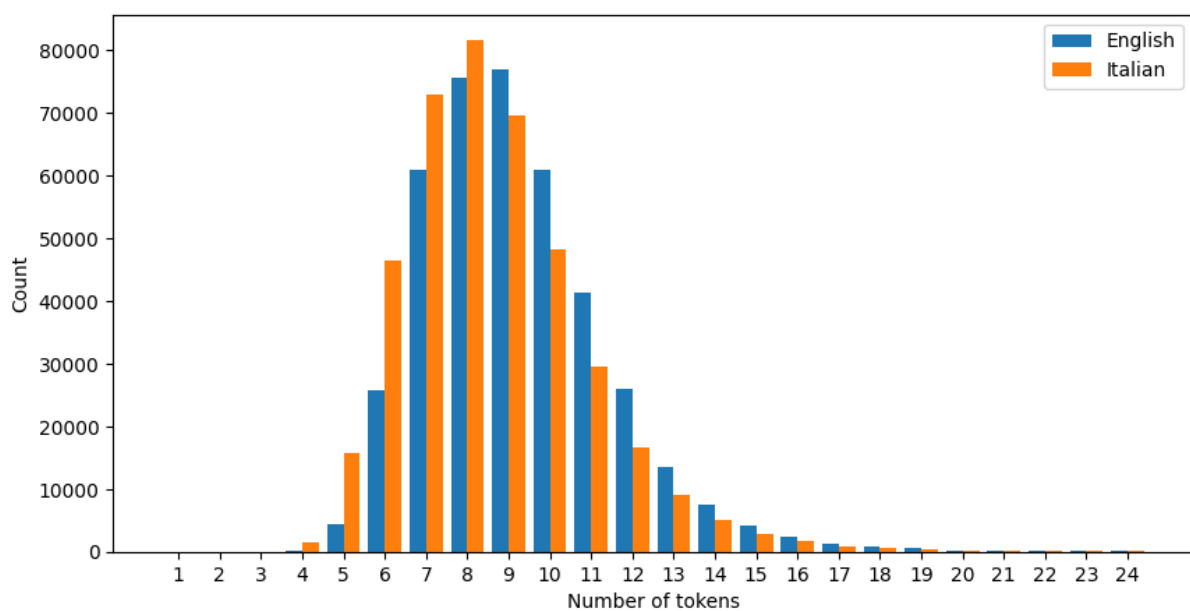
```
# Define the x-axis positions for the bars
x = np.arange(len(bins))

# Set the width of the bars
width = 0.4

# Plot the histogram
plt.figure(figsize=(10, 5))
plt.bar(x - width/2, eng_counts, width, label="English")
plt.bar(x + width/2, ita_counts, width, label="Italian")

# Customize the graph
plt.xticks(x, bins)
plt.xlabel("Number of tokens")
plt.ylabel("Count")
plt.legend()

# Show the plot
plt.show()
```



1.4.3 Vocabulary

Since the dataset consists of aligned sentence pairs in two different languages, we construct **two distinct vocabularies**: one for the source language and one for the target language. Un-

der a word-level tokenization scheme, the number of unique tokens can grow rapidly because each surface word form is treated as a separate vocabulary entry. To keep the vocabulary size manageable and reduce sparsity, tokens appearing fewer than two times in the corpus are replaced by a shared **unknown token** (). In addition to ordinary words, the vocabularies include several special-purpose tokens used to structure and control the sequence-processing pipeline. One important example is the **beginning-of-sequence token** (), which marks the start of a target sentence. During decoding, this token acts as the initial input to the model, signaling that generation of a new sequence should begin. Other special tokens include the **end-of-sequence token** (), already considered and the **padding token** (), which is used to standardize sequence lengths within a minibatch. If a sequence is shorter than the desired length, tokens are appended to its end until the required size is reached. Conversely, sequences that exceed the maximum allowed length are truncated by discarding tokens beyond the cutoff point. This procedure ensures that all sequences within the same minibatch share a common shape and can therefore be processed efficiently in parallel. Finally, we also record the true length of each source sequence excluding padding tokens. This information is often important during training and evaluation, since padding should not contribute to the model's representation or loss computation.

```
from collections import Counter

# Define special tokens to be included in the vocabulary
special_tokens=('<unk>', '<pad>', '<bos>', '<eos>')

def build_vocab(tokens, min_freq=2):

    # Flatten all sequences into a single list of tokens
    all_tokens = [token for sequence in tokens for token in sequence]

    # Remove special tokens
    filtered_tokens = [token for token in all_tokens if token not in special_tokens]

    # Count frequencies
    counter = Counter(filtered_tokens)

    # Identify tokens that meet the minimum frequency threshold
    frequent_tokens = [token for token, freq in counter.items() if freq >= min_freq]

    # Build the vocabulary by including special tokens and frequent tokens
    vocab = list(special_tokens) + frequent_tokens

    # Create mappings from tokens to indices and vice versa
```

```
token_to_idx = {token: idx for idx, token in enumerate(vocab)}  
idx_to_token = {idx: token for token, idx in token_to_idx.items()}  
  
# Return the token-to-index and index-to-token mappings  
return token_to_idx, idx_to_token
```

The function returns two complementary dictionaries: one maps tokens to integer indices, allowing text sequences to be converted into numerical representations suitable for the model, while the other performs the inverse mapping, transforming predicted indices back into readable text. Since the dataset contains two different languages, we construct one vocabulary pair for the source language and another for the target language:

```
# Build vocabularies for English and Italian tokens  
eng_to_idx, idx_to_eng = build_vocab(eng_sentences)  
ita_to_idx, idx_to_ita = build_vocab(ita_sentences)  
  
# Store the vocabularies size for later use  
eng_vocab_size = len(eng_to_idx)  
ita_vocab_size = len(ita_to_idx)  
  
# Show the sizes of the English and Italian vocabularies  
print(f"English vocabulary size: {eng_vocab_size}")  
print(f"Italian vocabulary size: {ita_vocab_size}")
```

English vocabulary size: 11726

Italian vocabulary size: 21442

These dictionaries allow us to convert a text sequence into a sequence of token IDs suitable for numerical processing, and conversely to reconstruct readable text from a sequence of predicted token IDs. We use also a length cutoff to limit the maximum sequence length, which is important for preparing later batches of data with uniform shape:

```
def encode(sequence, vocab, length):  
  
    # Get the indices for the unknown and padding tokens from the vocabulary  
    unk, pad = vocab['<unk>'], vocab['<pad>']  
  
    # Get ids for all tokens in the sequence,  
    # using the unknown token index for out-of-vocabulary tokens
```

```
ids = [vocab.get(t, unk) for t in sequence][:length]

# Pad the sequence with the padding token index
ids += [pad] * (length - len(ids))

# Return the list of token ids
return ids

def decode(indices, vocab):

    # Convert token indices back to tokens, ignoring padding tokens
    sequence = [vocab[i] for i in indices if vocab[i] != '<pad>']

    # Return the list of tokens corresponding to the input indices
    return sequence
```

For example, we can take a tokenized source sentence, convert it into a sequence of token IDs using the encode function, and then apply the inverse mapping to recover the original text:

```
# Generate a sample sequence of tokens and demonstrate encoding and decoding
tokens = ['<bos>', 'the', 'cat', '<eos>']
print(tokens)

# Encode the sample sequence into token indices using the English vocabulary
indices = encode(tokens, eng_to_idx, length=10)
print(indices)

# Decode the token indices back into tokens using the English vocabulary
decoded = decode(indices, idx_to_eng)
print(decoded)
```

```
['<bos>', 'the', 'cat', '<eos>']
[2, 845, 330, 3, 1, 1, 1, 1, 1, 1]
['<bos>', 'the', 'cat', '<eos>']
```

1.4.4 Pipeline

At this point we have all the individual ingredients in place: a preprocessed corpus, a tokenizer, and two vocabularies that map words to integer indices. We can assemble these pieces into a coherent data pipeline, one that can efficiently produce batches of training examples ready to be

consumed by a model. We will exploit the Dataset class (to define how individual examples are accessed and transformed) and DataLoader classes (to handle batching, shuffling, and parallel data loading) from PyTorch:

```
from torch.utils.data import Dataset

class TranslationDataset(Dataset):

    def __init__(self, sources, targets, source_vocab, target_vocab, length):

        # Store the encode sources
        self.sources = self._encode(sources, source_vocab, length)

        # Store the encoded targets
        self.targets = self._encode(targets, target_vocab, length)

        # Internal method to encode the list of sequences
        def _encode(self, sequences, vocab, length):

            # Encode each sequence in the list
            indices = [encode(seq, vocab, length) for seq in sequences]

            # Return the tensor of token indices
            return torch.tensor(indices, dtype=torch.long)

        # Return the number of examples in the dataset
        def __len__(self):
            return len(self.sources)

        # Return a specific example from the dataset
        def __getitem__(self, index):
            return self.sources[index], self.targets[index]
```

We instantiate the dataset from the full corpus and split it into a training set and a validation set:

```
from torch.utils.data import random_split

# Create a TranslationDataset instance for the English-Italian sentence pairs
translation_dataset = TranslationDataset(eng_sentences, ita_sentences, eng_to_idx,
    ↪ ita_to_idx, length=20)
```

```
# Define the sizes of training and testing sets (90% train, 10% testing)
train_size = int(0.9 * len(translation_dataset))
test_size = len(translation_dataset) - train_size

# Split the dataset into training and testing sets using random_split
translation_train_set, translation_test_set = random_split(translation_dataset,
    ↪ [train_size, test_size])

# Print the number of samples in the training and testing sets
print(f"Training samples: {train_size}")
print(f"Testing samples: {test_size}")
```

Training samples: 363129

Testing samples: 40348

Finally, we wrap each split in a DataLoader:

```
from torch.utils.data import DataLoader

# Create DataLoader instances for the training set
translation_train_loader = DataLoader(translation_train_set, batch_size=128,
    ↪ shuffle=True)

# Create DataLoader instances for the testing set
translation_test_loader = DataLoader(translation_test_set, batch_size=128,
    ↪ shuffle=False)
```

With the pipeline in place, sentences are now represented as a sequence of vectors, a numeric format that a machine learning model can operate on.

1.4.5 Word embeddings

However, token indices are merely arbitrary integer identifiers and carry no intrinsic geometric or semantic structure. In the space of integers, token 42 is not meaningfully "closer" to token 41 than to token 9000. Since neural networks operate through algebraic operations (matrix multiplications, weighted sums, distances, dot products) and these operations implicitly assume that the numerical inputs possess some meaningful geometric structure. Raw token IDs do not satisfy this assumption. An **embedding layer** addresses this problem by associating each token with a **learnable dense vector representation**. Concretely, the layer maintains a parameter matrix:

$$E \in \mathbb{R}^{V \times D}$$

where V is the vocabulary size and D is the embedding dimension. Each row i is the vector representation of the token with index i . Looking up the embedding for a token is simply selecting the corresponding row:

$$e(\text{token}_i) = E[i] \in \mathbb{R}^D$$

The embedding vectors are typically initialized randomly and optimized jointly with the rest of the network during training. Over time, the model learns a geometry in which tokens appearing in similar linguistic contexts acquire similar vector representations. This is a simple implementation:

```
class Embedding(nn.Module):
    def __init__(self, vocab_size, embed_dim):
        super().__init__()

        # Initialize the embedding matrix as a learnable parameter
        self.weights = nn.Parameter(torch.randn(vocab_size, embed_dim))

    # idx : (batch, seq_len)
    def forward(self, idx):

        # Look up the embedding vectors for the given token indices
        return self.weights[idx] # (batch, seq_len, embed_dim)
```

As a simple example, we instantiate an embedding layer and use it to retrieve the vector representations associated with two tokens from the vocabulary. Initially, these vectors are random, since the embedding matrix has not yet been trained, but during optimization the model will gradually organize them into a meaningful geometric representation space.

```
# Select the dimensionality of the embedding space
embed_dim = 4

# Create an embedding layer
embedding = Embedding(eng_vocab_size, embed_dim)
print("Embedding matrix shape:", embedding.weights.shape)

# Look up embeddings for two tokens using the existing vocabulary
cat_id = torch.tensor([eng_to_idx["cat"]])
dog_id = torch.tensor([eng_to_idx["dog"]])
```

```
cat_embedded = embedding(cat_id)
dog_embedded = embedding(dog_id)

print("Embedding of 'cat':", cat_embedded)
print("Embedding of 'dog':", dog_embedded)
```

```
Embedding matrix shape: torch.Size([11726, 4])
Embedding of 'cat': tensor([[ 0.4303,  0.3305,  1.3761, -0.7184]], grad_fn=<IndexBackward0>)
Embedding of 'dog': tensor([[ -1.8478,  0.4687,  0.9946, -0.6954]], grad_fn=<IndexBackward0>)
```

For example, after training, we would generally expect the embedding vectors associated with semantically related words such as “cat” and “dog” to lie closer to one another in the embedding space than to the vector associated with an unrelated concept such as “run”. Of course we need two separate embedding layers for the source and target languages, since they have different vocabularies and therefore require different vector representations. The source embedding layer maps English tokens to vectors, while the target embedding layer maps Italian tokens to vectors. These embeddings serve as the initial input representations for the model, providing a dense and structured encoding of the discrete token sequences that can be processed by subsequent layers:

```
# Define embedding dimensions
embed_dim = 256

# Create separate embedding layers for English and Italian vocabularies
eng_embedding = Embedding(eng_vocab_size, embed_dim)
ita_embedding = Embedding(ita_vocab_size, embed_dim)
```

1.4.6 Evaluation

Defining a problem also means **deciding how to measure success**. For machine translation, this is less straightforward than it might seem: a sentence can have many valid translations, and comparing two strings character by character would penalise perfectly correct outputs that simply chose different words. The standard metric is **Bilingual Evaluation Understudy (BLEU)**, which measures how much of the predicted output appears in the reference translation, at multiple levels of granularity. The key building block is the notion of an **n-gram**, a contiguous subsequence of exactly n tokens. For example, given the sentence:

“the cat is on the mat”

Its 1-grams (unigrams), 2-grams (bigrams), and 3-grams (trigrams) are:

- 1-grams: "the", "cat", "is", "on", "the", "mat"
- 2-grams: "the cat", "cat is", "is on", "on the", "the mat"
- 3-grams: "the cat is", "cat is on", "is on the", "on the mat"

Longer n-grams **capture more context** and impose **stricter ordering constraints**: matching a 4-gram requires getting four consecutive words right, not just individual tokens. The core idea of BLEU is to measure, for each order n, how many n-grams in the predicted sequence also appear in the reference:

$$p_n = \frac{\sum_{s \in G_n(\hat{y})} \min(c(s, \hat{y}), c(s, y))}{\sum_{s \in G_n(\hat{y})} c(s, \hat{y})}$$

where G_n is the set of distinct n-grams in a sentence and $c()$ counts how many times an n-gram occurs. The minimum operation in the numerator is a **clipping mechanism**: without it, a model could inflate the score by repeating a common word many times ("the the the the"). Clipping ensures that a predicted n-gram is credited at most as many times as it appears in the reference. Here an implementation in Python:

```
def ngram_precision(prediction, target, n):

    # Extract all n-grams from prediction and target
    pred_ngrams = Counter(tuple(prediction[i:i+n]) for i in range(len(prediction) -
↪ n + 1))
    target_ngrams = Counter(tuple(target[i:i+n]) for i in range(len(target) -
↪ n + 1))

    # For each predicted n-gram, count how many times it appears in the target
    # (capped at the number of times it appears in the target, to avoid double
    ↪ counting)
    matched = sum(min(pred_count, target_ngrams[ngram]) for ngram, pred_count in
↪ pred_ngrams.items())

    # Precision = matched n-grams / total predicted n-grams
    total = sum(pred_ngrams.values())
    precision = matched / total if total > 0 else 0.0

    # Return the precision
    return precision
```

Taking the earlier example, where the reference is "the cat is on the mat" and the prediction is "the cat is on the roof", the precisions at each order are:

n	Matched n-grams	p_n
1	"the", "cat", "is", "on", "the" → 5 of 6	5/6
2	"the cat", "cat is", "is on", "on the" → 4 of 5	4/5
3	"the cat is", "cat is on", "is on the" → 3 of 4	3/4
4	"the cat is on", "cat is on the" → 2 of 3	2/3

The scores drop as n grows because the single mismatch at the end ("roof" instead of "mat") contaminates progressively longer windows. Precision alone has a **blind spot**: it rewards every correct word but never asks what was missed. A prediction of just "the" scores a perfect unigram precision against almost any reference, despite conveying nothing. The natural fix would be recall, but recall is awkward when multiple valid translations of different lengths exist. BLEU instead applies a **brevity penalty** that discounts scores when the prediction is suspiciously short:

$$BP = \begin{cases} 1 & \text{if } |\hat{y}| \geq |y| \\ \exp\left(1 - \frac{|y|}{|\hat{y}|}\right) & \text{if } |\hat{y}| < |y| \end{cases}$$

There is no penalty when the prediction is at least as long as the reference. Below that threshold the penalty decays smoothly toward zero: the shorter the prediction, the harsher the discount.

```
import math

def brevity_penalty(predictions, targets):

    # Compute the total length of the predicted and target sequences
    pred_len = sum(len(pred) for pred in predictions)
    target_len = sum(len(tgt) for tgt in targets)

    # If the predicted length is greater than or equal to the target length,
    # return a penalty of 1.0 (no penalty)
    if pred_len >= target_len:
        return 1.0

    # Otherwise, return a penalty that decays exponentially as the predicted
    # length falls short of the target length
    return math.exp(1 - target_len / pred_len)
```

Combining the brevity penalty with precisions across all orders gives the final BLEU score:

$$\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^N \frac{1}{N} \log p_n\right)$$

The precisions are combined as a geometric mean rather than an arithmetic mean. Unlike the arithmetic mean, which averages values additively, the geometric mean multiplies them, so a single near-zero precision drags the whole product toward zero, no matter how well the other orders perform. This ensures that a catastrophically low score at any single order drags the overall score down significantly, so the model cannot ignore any level of matching.

```
def bleu(predictions, targets, N=4):

    # Compute the brevity penalty for the predictions and targets
    bp = brevity_penalty(predictions, targets)

    # Compute the precision for each n-gram order and store them in a list
    precisions = []
    for n in range(1, N + 1):
        pn = sum(ngram_precision(pred, tgt, n) for pred, tgt in zip(predictions,
↪ targets)) / len(predictions)
        precisions.append(pn)

    # Compute the logarithm of the precisions
    p_log_sum = sum((1 / N) * math.log(pn) if pn > 0 else float('-inf') for pn in
↪ precisions)

    # Combine the brevity penalty with the geometric mean of the precisions to get the
↪ final BLEU score
    return bp * math.exp(p_log_sum)
```

A BLEU score of 1 indicates a perfect match, a score of 0 means no overlap at any order. Short n-grams capture **vocabulary accuracy**, longer ones capture whether the model preserves **correct phrasing**, so a good BLEU score requires getting both individual words and their local ordering right.

With the problem fully specified (data, preprocessing, and evaluation metric) the remaining question is **what model to use**. We start with a recurrent encoder–decoder, and then introduce attention to overcome its limitations.

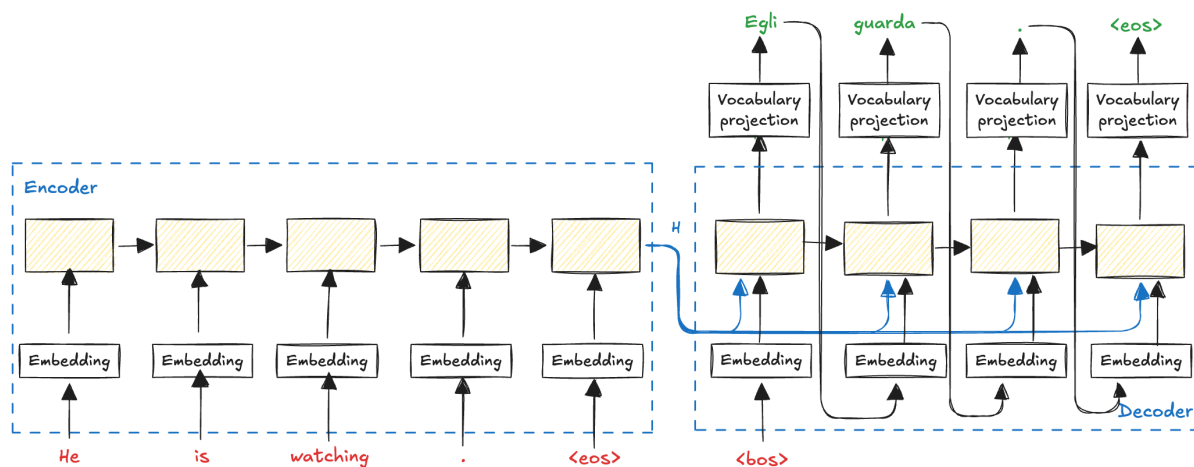
1.5 An Encoder–Decoder solution

The **encoder–decoder architecture**, introduced for sequence-to-sequence learning, consists of two networks working in tandem. The **encoder** reads the input sequence and compresses it

into an internal representation. The **decoder** then generates the output sequence one token at a time, using that representation together with the tokens it has already produced, as illustrated below:



For instance, given the English input ["He", "is", "watching", "."], the encoder compresses it into a single representation, and the decoder uses it to produce the Italian translation token by token ["Egli", "guarda", "."] as shown below:



The graph is unrolled in time, the encoder processes one word at a time, left to right. Each box in the encoder represents the same GRU cell at a different time step, not a separate module. So "He" goes in first, updates the hidden state, then "is" goes in and updates it again, and so on until . At the end, the final hidden state is passed to the decoder. The same applies to the decoder: each box is the same GRU cell unrolled over time, starting from and generating one token per step. Now, let's consider how to implement this architecture concretely.

1.5.1 Encoder

The encoder reads the input sequence:

$$x_1, x_2, \dots, x_T$$

and at each time step updates its hidden state:

$$h_t = f(x_t, h_{t-1})$$

A **context variable** is then derived from the full collection of hidden states to be passed to the decoder:

$$c = q(h_1, h_2, \dots, h_T)$$

The simplest choice, and the one used here, is to take the final hidden state as the context:

$$c = h_T$$

This **compresses** the entire input sequence into a **single fixed-size vector**. More expressive alternatives exist, for instance, bidirectional RNNs process the sequence in both directions, giving each position access to both past and future context. However, for clarity we proceed with the simplest formulation. In the following implementation, the encoder stacks our custom embedding layer and a multi-layer GRU:

```
class Encoder(nn.Module):

    def __init__(self, embedding, hidden_dim, num_layers, dropout=0.0):
        super().__init__()

        # Store the embedding layer to convert token IDs into dense vectors
        self.embedding = embedding

        # Create a multi-layer GRU to process the embedded input sequence and produce
        #   hidden states
        self.rnn = nn.GRU(embed_dim,          # Input size: the dimension of the word
                           embeddings        # embeddings
                           hidden_dim,       # The size of the hidden state in the GRU
                           num_layers,      # The number of stacked GRU layers to use
                           batch_first=True, # Input shape is (batch, seq_len, embed_dim)
                           dropout=dropout) # Dropout applied between GRU layers

        # in the encoder

    def forward(self, sequence):

        # Embed the input sequence of token IDs into dense vectors
        emb = self.embedding(sequence)

        # Pass the embedded sequence through the GRU to obtain the context and hidden
        #   states
        output, state = self.rnn(emb)

        # Return the output: the sequence of hidden states for each time step
        # And the final state
        return output, state
```

1.5.2 Decoder

The decoder starts from the encoder's final hidden state, which carries a compressed summary of the entire source sentence. From there it generates the translation one token at a time: at each step it takes the token it just produced (or the special <bos> token at the very beginning), converts it to a dense vector, and feeds it into the GRU along with the current hidden state. A linear layer then projects the output onto the target vocabulary to get a score for each possible next word, and the highest-scoring word becomes the next token, which is immediately fed back as input for the following step:

```
class Decoder(nn.Module):

    def __init__(self, embedding, hidden_dim, num_layers, dropout=0.0):
        super().__init__()

        # Store the embedding layer to convert token IDs into dense vectors
        self.embedding = embedding

        # Create a multi-layer GRU to process the embedded input sequence and produce
        #   hidden states
        self.rnn = nn.GRU(embed_dim, hidden_dim, num_layers,
                           batch_first=True,
                           dropout=dropout)

        # Create a linear layer to project the hidden states to the vocabulary size
        # for output prediction
        self.fc = nn.Linear(hidden_dim, ita_vocab_size)

    def forward(self, Y, state):

        # Embed the input sequence of token IDs into dense vectors
        emb = self.embedding(Y)

        # Pass the embedded sequence through the GRU using the provided hidden state
        output, state = self.rnn(emb, state)

        # Project the GRU output onto the vocabulary size to get logits for the next
        #   token prediction
        logits = self.fc(output)

        # Return the logits for the predicted next tokens and the updated hidden state
        ↵
```

```
return logits, state
```

1.5.3 Seq2seq model

We combine the encoder and decoder into a single Seq2Seq module that wires them together: it runs the encoder on the source sequence to get the final hidden state, then passes it to the decoder. Notice that **during inference**, the decoder operates **autoregressively**: starting from the <bos> token, it generates one token at a time, feeding each newly produced token back as input to predict the next one. This continues until the decoder emits the <eos> token. During training, however, we already know the correct target sequence, so instead of feeding back the model's own (possibly wrong) predictions, we feed the actual target tokens at each step, a strategy known as **teacher forcing**. Concretely, if the target is

[<bos>, w1, w2, w3, <eos>]

the decoder receives

[<bos>, w1, w2, w3] as input and is trained to predict

[w1, w2, w3, <eos>]

at each position. Since all input tokens are known upfront, the GRU can process the entire sequence in one forward pass rather than step by step, making training significantly more efficient.

```
class Seq2Seq(nn.Module):

    def __init__(self, encoder, decoder):
        super().__init__()

        # Store the encoder
        self.encoder = encoder

        # Store the decoder
        self.decoder = decoder

    def forward(self, X, Y):

        # Run the encoder on the input sequence X to get the final hidden state
        _, state = self.encoder(X)

        # Run the decoder on the target sequence Y using the encoder's final hidden
        ↪ state
```

```
logits, _ = self.decoder(Y, state)

# Return the logits for the predicted next tokens
return logits
```

We can now instantiate the encoder and decoder separately, each with its own embedding and combine them into the final Seq2Seq model:

```
# Instantiate the encoder
encoder = Encoder(eng_embedding, hidden_dim=512, num_layers=2, dropout=0.3)

# Instantiate the decoder
decoder = Decoder(ita_embedding, hidden_dim=512, num_layers=2, dropout=0.3)

# Instantiate the seq2seq model
seq2seq_model = Seq2Seq(encoder, decoder)
```

1.5.4 Loss function

At each decoding step, the model outputs a vector of logits:

$$z \in \mathbb{R}^V$$

over the target vocabulary, from which a probability distribution is obtained via softmax:

$$p_j = \frac{\exp(z_j)}{\sum_{k=1}^V \exp(z_k)}$$

Predicting the correct next token out of several candidates is a **multi-class classification problem**, making **cross-entropy loss** the natural training objective. At each position, it penalises the model by how little probability mass it assigned to the correct token:

$$\ell = -\log p_y = -z_y + \log \sum_{k=1}^V \exp(z_k)$$

A practical complication arises because sentences in the same minibatch generally have different lengths. To enable efficient batch processing, shorter sequences are **padded** with the special "<pad>" token so that all sequences share a common length. For example, if a batch contains the target sentences:

["Amo", "i", "gatti"]

["Il", "cane", "corre", "velocemente"]

the shorter sentence is padded as:

["Amo", "i", "gatti", "<pad>"]

Padding positions carry no meaningful prediction target, so including them in the loss would dilute the gradient signal. They must be excluded, so that the final loss averages only over the positions containing real tokens:

$$\mathcal{L} = -\frac{1}{|\{(b,t) : y_{b,t} \neq \text{pad}\}|} \sum_b \sum_{t: y_{b,t} \neq \text{pad}} \log p_{y_{b,t}}$$

We can implement this loss function in Python as follows:

```
def cross_entropy_loss(logits, target):

    # Create a mask to ignore padding tokens in the loss computation
    mask = target != ita_to_idx['<pad>']

    # Compute the log probabilities of the predicted logits
    log_probs = F.log_softmax(logits, dim=-1)

    # Gather the log probabilities corresponding to the correct target tokens
    correct = log_probs.gather(-1, target.unsqueeze(-1)).squeeze(-1)

    # Return the negative mean log probability of the correct tokens,
    # averaged only over the non-padding tokens
    return -correct[mask].mean()
```

We jointly optimize all parameters of both the encoder and decoder using the Adam optimizer, a widely adopted default choice that automatically adapts the effective learning rate for each parameter during training. We use a standard initial learning rate:

```
# Create an optimizer for the seq2seq model parameters
seq2seq_optimizer = torch.optim.Adam(seq2seq_model.parameters(), lr=1e-3)
```

Before starting training, it is often useful to compute the **total number of trainable parameters**, all weights and biases updated by the optimizer during learning. This provides a rough indication of the **model's representational capacity** as well as its **computational and memory requirements**:

```
# Compute the total number of trainable parameters in the seq2seq model
total_params = sum(p.numel() for p in seq2seq_model.parameters() if p.requires_grad)

# Print the total number of trainable parameters
print(f"Model parameters: {total_params:,}")
```

Model parameters: 25,008,066

With roughly 25 million trainable parameters, the model is **moderately sized**. A substantial fraction of these parameters comes from the two embedding layers (one for each language) together with the weights of the stacked GRU layers. This scale is large enough to capture meaningful translation patterns and nontrivial linguistic structure in the dataset, while still remaining compact enough to train in a reasonable amount of time on a modest GPU.

1.5.5 Training loop

Before starting training, we check whether an accelerated computation device is available so that the model can be moved to a GPU or, on Apple Silicon systems, to an MPS backend for faster execution:

```
# Check for available device (GPU, MPS, or CPU)
device = torch.device(
    'cuda' if torch.cuda.is_available() else
    'mps'  if torch.backends.mps.is_available() else
    'cpu'
)

# Send the model to the selected device
seq2seq_model = seq2seq_model.to(device)

# Show the selected device
print(f"Using device: {device}")
```

Using device: mps

The training loop iterates over minibatches, performs a forward pass, computes the loss, clips gradients to prevent the instability common in recurrent networks, and updates the parameters. It returns the average loss over the epoch. Notice that it should prepare the

```
def train(model, loader, optimizer):

    # Set the model to training mode
    model.train()

    # Initialize a variable to accumulate the total loss across all batches
    total_loss = 0

    # Iterate over the batches of data provided by the loader
    for source, target in loader:

        # Move the source and target tensors to the selected device
        source = source.to(device)
        target = target.to(device)

        # Split target into decoder input (all but last token)
        # and decoder output (all but first token)
        dec_in = target[:, :-1]
        dec_out = target[:, 1:]

        # Clear the gradients of all model parameters to prepare for the backward pass
        optimizer.zero_grad()

        # Perform a forward pass through the model to obtain the predicted logits for
        # the current batch
        logits = model(source, dec_in)

        # Compute the loss by comparing the predicted logits with the actual decoder
        # output
        loss = cross_entropy_loss(logits, dec_out)

        # Perform a backward pass to compute the gradients of the loss with respect
        # to all model parameters
        loss.backward()

        # Clip the gradients to a maximum norm of 1.0 to prevent exploding gradients
        # during training
        nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

        # Update the model parameters using the optimizer based on the computed
        # gradients
        optimizer.step()
```

```
# Accumulate the loss for the current batch into the total loss variable
total_loss += loss.item()

# After processing all batches, compute the average loss by dividing the total
# loss by
# the number of batches
return total_loss / len(loader)
```

The evaluation function mirrors the training loop but with two key differences: `model.eval()` switches off dropout and other training-specific behaviours, and `torch.no_grad()` disables gradient computation entirely (since no weight update is needed) making the validation pass faster and more memory-efficient. During evaluation we would ideally use the model's own predictions autoregressively, that's the honest way to measure performance. However, using teacher forcing also in evaluation is a common simplification that makes the loss computation much easier to implement: we just do one forward pass over the whole sequence and get a number back. The downside is that the loss we measure is **slightly optimistic** compared to real inference, because the decoder never has to recover from its own mistakes. The real quality of the model is better captured by metrics like BLEU, which are computed on actual autoregressive translations rather than on teacher-forced ones. So it's a practical compromise: teacher forcing during evaluation gives us a fast computation to monitor training, while BLEU gives us the true picture of translation quality at the end.

```
def evaluate(model, loader):

    # Set the model to evaluation mode
    model.eval()

    # Initialize a variable to accumulate the total loss across all batches
    total_loss = 0

    # Disable gradient computation since we are only evaluating the model
    with torch.no_grad():

        # Iterate over the batches of data provided by the loader
        for source, target in loader:

            # Move tensors to the selected device
            source = source.to(device)
            target = target.to(device)
```

```
# Split target into decoder input (all but last token)
# and decoder output (all but first token)
dec_in = target[:, :-1]
dec_out = target[:, 1:]

# Perform a forward pass through the
logits = model(source, dec_in)

# Compute the loss by comparing the predicted logits with the actual
# decoder output
loss = cross_entropy_loss(logits, dec_out)

# Accumulate the loss for the current batch into the total loss variable
total_loss += loss.item()

# After processing all batches, compute the average loss by dividing the total
# loss by
return total_loss / len(loader)
```

We define a simple training loop that runs for a specified number of epochs, calling the training and evaluation functions at each step and printing the resulting losses. Since training can be time-consuming, we also implement a caching mechanism to save the model's state to disk after the first run. This way, if we need to rerun the code later, we can skip the training phase and directly load the saved checkpoint, saving time and computational resources.

```
import os

def train_model(model, optimizer, model_path):

    # Initialize lists to store training and validation losses for each epoch
    train_losses, val_losses = [], []

    # Check if a model checkpoint exists at the specified path
    if os.path.exists(model_path):
        # Load the model checkpoint from the file, mapping it to the selected device
        checkpoint = torch.load(model_path, map_location=device)
        model.load_state_dict(checkpoint["model_state"])
        train_losses = checkpoint["train_losses"]
        val_losses = checkpoint["val_losses"]
        model.eval()
        print(f"Model loaded from {model_path}")
```

```
# Otherwise, if no checkpoint exists, run the training loop and save the model to
↳ disk
else:

    # Run the training loop for the specified number of epochs
    for epoch in range(1, num_epochs + 1):

        # Perform one epoch of training and compute the training loss
        train_loss = train(model, translation_train_loader, optimizer)

        # Perform evaluation on the validation set and compute the validation loss
        val_loss = evaluate(model, translation_test_loader)

        # Append the training and validation losses for the current epoch to their
        ↳ respective lists
        train_losses.append(train_loss)
        val_losses.append(val_loss)

        # Provide a real-time update of the training progress
        print(f"Epoch {epoch:2d} | train loss {train_loss:.4f} | val loss
        ↳ {val_loss:.4f}", end='\r', flush=True)

    # Save the trained model's state along with the training and validation losses
    # to a checkpoint file for future use
    torch.save({
        "model_state": model.state_dict(),
        "train_losses": train_losses,
        "val_losses": val_losses,
    }, model_path)
    print(f"\nModel saved to {model_path}")

return train_losses, val_losses
```

We apply this training loop to our Seq2Seq model:

```
# Define the number of epochs for training
num_epochs = 10

# Define the path to the model checkpoint file
model_path = "./data/seq2seq_model.pt"
```

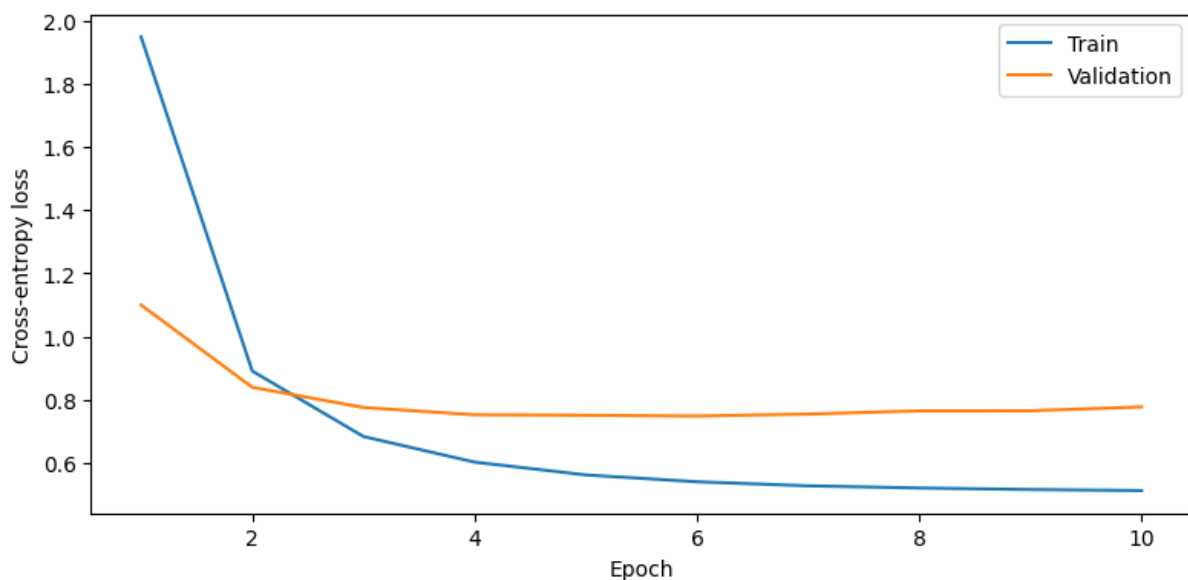
```
# Train the model and obtain the training and validation losses for each epoch
seq2seq_train_losses, seq2seq_val_losses = train_model(seq2seq_model,
    ↪ seq2seq_optimizer, model_path)
```

Epoch 10 | train loss 0.5121 | val loss 0.7768

Model saved to ./data/seq2seq_model.pt

We plot the learning curves (training and validation loss over epochs) to visually diagnose the training dynamics: ideally both curves should decrease steadily and converge close together; a large and growing gap between them would signal **overfitting**, while both curves remaining high would suggest **underfitting**:

```
plt.figure(figsize=(8, 4))
plt.plot(range(1, num_epochs + 1), seq2seq_train_losses, label='Train')
plt.plot(range(1, num_epochs + 1), seq2seq_val_losses, label='Validation')
plt.xlabel('Epoch')
plt.ylabel('Cross-entropy loss')
plt.legend()
plt.tight_layout()
plt.show()
```



Both losses drop sharply in the first two epochs, then the training loss continues to decrease slowly and steadily while the validation loss flattens. The growing gap between the two curves is a mild sign of overfitting (the model keeps improving on the training data but stops generalising

beyond epoch 3 or so). Techniques such as early stopping, stronger dropout, or reducing the model capacity could help close this gap.

1.5.6 Prediction

During inference, the target sequence is not available, so the decoder must operate autoregressively as introduced earlier: starting from <bos>, it feeds each predicted token back as input for the next step, selecting at each step the highest-scoring word from the vocabulary projection. The process stops as soon as the decoder emits <eos>:

```
def seq2seq_predict(source, max_len=20):

    # Set the model to evaluation mode
    seq2seq_model.eval()

    # Disable gradient computation since we are only performing inference
    with torch.no_grad():

        # Get the indices for the padding, beginning-of-sequence, and unknown tokens
        pad = eng_to_idx['<pad>']
        bos = ita_to_idx['<bos>']
        unk = eng_to_idx['<unk>']

        # Wrap the source sentence with <bos> and <eos> to match training format
        source = ['<bos>'] + source + ['<eos>']

        # Encode the source sentence
        src_ids = [eng_to_idx.get(token, unk) for token in source][:max_len]

        # Pad the source sentence with the padding token
        src_ids += [pad] * (max_len - len(src_ids))

        # Convert the list of source token ids into a tensor and move it to the
        ↪ selected device
        src = torch.tensor(src_ids, dtype=torch.long).unsqueeze(0).to(device)

        # Run the encoder on the source sentence to get the initial context for
        ↪ decoding
        _, context = seq2seq_model.encoder(src)

        # Initialize the decoder input with the beginning-of-sequence token
        token = torch.tensor([[bos]], dtype=torch.long).to(device)
```

```
# Initialize a list to store the predicted output tokens
output = []

# Iteratively generate tokens until the end-of-sequence token is produced
# or the maximum length is reached
for _ in range(max_len):

    # Perform a forward pass through the decoder using the current token and
    # context
    # to get the logits for the next token prediction
    logits, context = seq2seq_model.decoder(token, context)

    # Select the token with the highest predicted probability as the next token
    token = logits.argmax(dim=-1)

    # Convert the predicted token index back to a word using the Italian
    # vocabulary
    word = idx_to_ita[token.item()]

    # If the predicted word is the end-of-sequence token, stop generating
    # further tokens
    if word == '<eos>':
        break

    # Append the predicted word to the output list
    output.append(word)

# Return the list of predicted output tokens
return output
```

We evaluate the model on a few hand-picked English sentences and inspect the Italian output:

```
# Examples to inspect the predictions of the trained seq2seq model
examples = [
    ["i", "love", "you", "."],
    ["the", "cat", "is", "on", "the", "table", "."],
    ["where", "is", "the", "train", "station", "?"],
    ["good", "morning", "!"],
    ["she", "reads", "a", "book", "every", "day", "."],
    ["we", "are", "happy", "."],
    ["what", "time", "is", "it", "?"],
```

```
["can", "you", "help", "me", "?"],  
["this", "is", "a", "test", "."],  
]  
  
# For each example sentence, use the seq2seq_predict function to generate a  
# translation and print the original English sentence along with the  
# predicted Italian translation  
for src in examples:  
    translation = seq_2_seq_predict(src)  
    print(f" EN: {' '.join(src)}")  
    print(f" IT: {' '.join(translation)}")  
    print()
```

EN: i love you .

IT: io vi amo .

EN: the cat is on the table .

IT: il gatto è sdraiato sul tavolo .

EN: where is the train station ?

IT: dov ' è la stazione ferroviaria ?

EN: good morning !

IT: buongiorno !

EN: she reads a book every day .

IT: lei legge un libro ogni giorno .

EN: we are happy .

IT: siamo felici .

EN: what time is it ?

IT: che ora è ?

EN: can you help me ?

IT: mi potete aiutare ?

EN: this is a test .

IT: questo è un test .

1.5.7 Performance

We define a helper function that runs the model over the entire test set, computes an overall BLEU score, and then breaks it down by source sentence length so we can see how performance varies as sentences get longer.

```
def evaluate_bleu(model, predict_fn):

    # Initialize lists to store the target sentences, predicted sentences,
    # and their lengths
    targets      = []
    predictions  = []
    lengths      = []

    # Iterate over the indices of the test set to evaluate the model's predictions
    # against the reference translations
    for idx in translation_test_set.indices:

        # Strip <bos> and <eos> from source
        src = eng_sentences[idx][1:-1]

        # Strip <bos> and <eos> from reference
        ref = ita_sentences[idx][1:-1]

        # Generate a prediction for the source sentence
        pred = predict_fn(src)

        # Append the reference translation, predicted translation,
        # and source sentence length to their respective lists
        targets.append(ref)
        predictions.append(pred)
        lengths.append(len(src))

    # Compute the overall BLEU score for all predictions and targets
    bleu_overall = bleu(predictions, targets, N=4)

    # Compute BLEU scores for predictions of different source sentence lengths
    bleu_by_length = {}

    # Iterate over the unique sentence lengths in the test set
```

```
for l in sorted(set(lengths)):

    # Filter the references and predictions to include only those
    # with the current sentence length
    refs_l = [ref for ref, length in zip(targets, lengths) if length == l]
    preds_l = [pred for pred, length in zip(predictions, lengths) if length == l]

    # Compute the BLEU score for the current sentence length
    # and store it in the dictionary
    bleu_by_length[l] = bleu(preds_l, refs_l, N=4)

# Return the overall BLEU score and the BLEU scores by sentence length
return bleu_overall, bleu_by_length
```

Then we evaluate the model on the validation set:

```
# Evaluate the BLEU score of the trained seq2seq model on the test set and print the
↪ overall BLEU score
seq2seq_bleu_overall, seq2seq_bleu_by_length = evaluate_bleu(seq2seq_model,
↪ seq_2_seq_predict)

# Print the overall BLEU score for the seq2seq model
print((f"Seq2Seq Overall BLEU: {seq2seq_bleu_overall:.4f}\n"))
```

Seq2Seq Overall BLEU: 0.4960

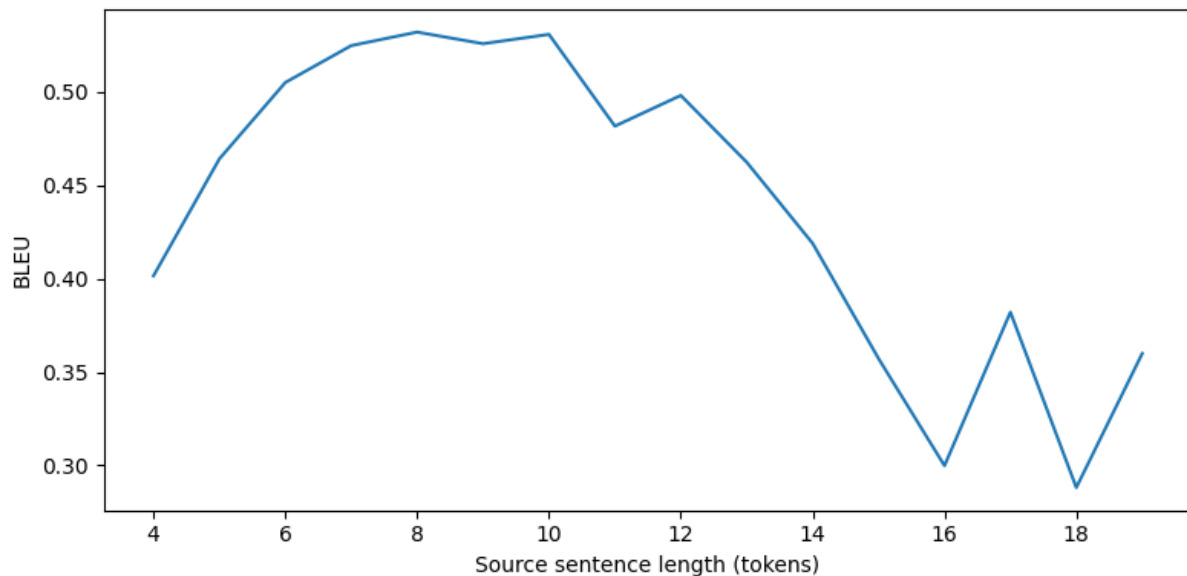
That BLEU means roughly half the predicted n-grams match the reference, a solid baseline for a simple RNN trained on short sentence pairs, and a useful starting point before moving to more expressive architectures. However we can inspect how the performance varies with sentence length:

```
# Plot BLEU scores by source sentence length, only for lengths
# available in the dataset (see the histogram)
lengths = [l for l in sorted(seq2seq_bleu_by_length.keys()) if 3 < l < 20]

# Extract the corresponding BLEU scores for the selected lengths
seq2seq_bleus = [seq2seq_bleu_by_length[l] for l in lengths]

# Create a line plot of BLEU scores by source sentence length
```

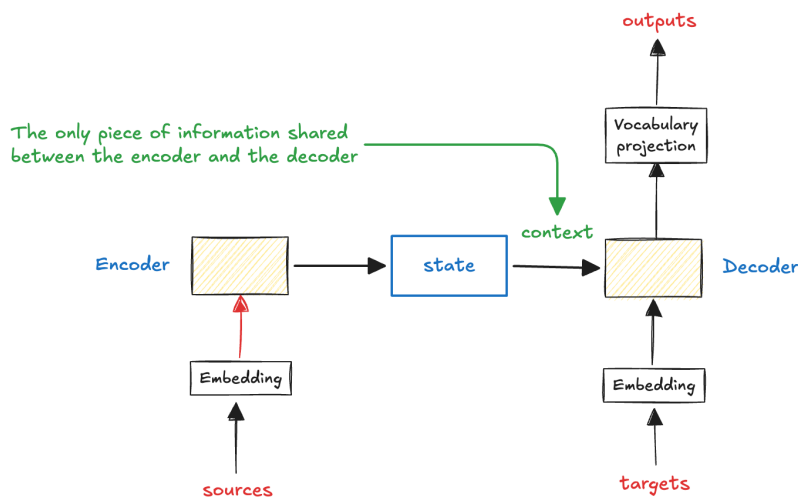
```
plt.figure(figsize=(8, 4))
plt.plot(lengths, seq2seq_bleus)
plt.xlabel('Source sentence length (tokens)')
plt.ylabel('BLEU')
plt.tight_layout()
plt.show()
```



We can notice how translation quality degrades as sentences get longer. This is a common issue with encoder–decoder models that rely on a single fixed-size context vector: as the input sequence grows, it becomes increasingly difficult for the encoder to compress all relevant information into that single representation, and the decoder struggles to generate accurate outputs based on an incomplete summary of the input. This motivates the introduction of attention mechanisms, which allow the decoder to access the entire sequence of encoder hidden states rather than relying solely on a single context vector.

1.6 Overcoming the bottleneck

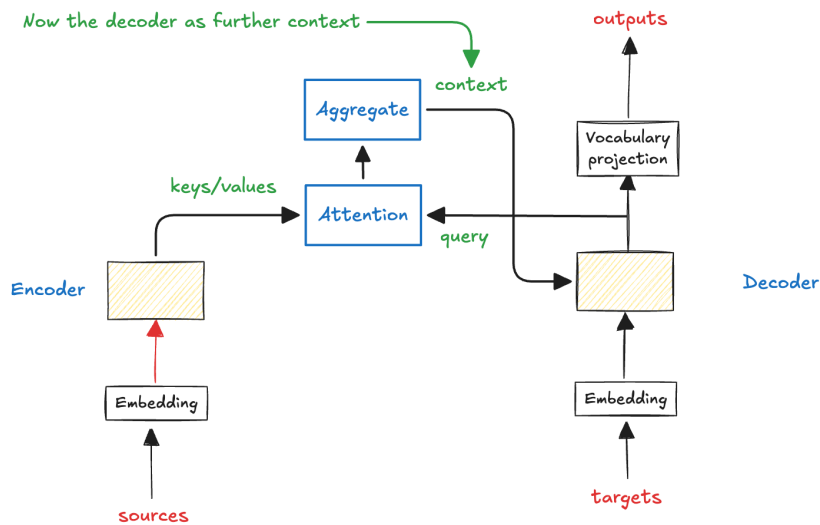
The encoder–decoder architecture suffers from a **fundamental limitation**. In its basic form, the encoder compresses all the information contained in the input sequence into a single fixed-dimensional representation. This representation is then passed to the decoder, which relies on it as the sole source of information for generating the output sequence. In this sense, the intermediate state is implicitly treated as a **sufficient statistic** of the entire input:



While this abstraction is **reasonable when the input is short** and the amount of information to be retained is limited, it becomes problematic as the input grows in length or complexity. It arises whenever the input is structured, variable in size, or information-rich, such as long sentences, audio signals, or high-resolution images. A fixed-size representation must compress all relevant details into a limited number of dimensions, inevitably leading to information loss. As a result, the decoder is forced to **operate on an incomplete summary of the input**, which can degrade performance.

1.6.1 Adding attention

This limitation naturally motivates the introduction of **attention mechanisms**, whose purpose is to learn an explicit and dynamic notion of alignment between input and output. When humans translate a sentence, they do not rely on a single compressed summary of the entire source. Instead, while generating each word, they **selectively focus on the parts most relevant at that particular moment**, shifting toward the corresponding verb in the source when producing a verb, or toward the related noun when generating one. Translation therefore involves a **continuously evolving, context-dependent correspondence** between positions in the input and positions in the output. Rather than forcing the decoder to generate the entire translation from a single fixed-size vector, it is possible to apply attention mechanisms that allow the decoder to access the entire sequence of encoder hidden states. In this way, the model is allowed to revisit the entire input sequence at every decoding step.



At each decoding step, the decoder's current hidden state acts as the **query**, since it encodes what the model is currently looking for. The encoder hidden states serve as both **keys**, used to compute relevance scores against the query, and **values**, the actual content to be aggregated. The resulting attention weights define a **soft alignment** between source and target, and are used to construct a **context vector**, a weighted sum of the values, concentrating the information from the input positions most relevant to the current prediction. Concretely, instead of a fixed context variable:

$$c = h_T$$

with attention, the state becomes dynamic, at each decoding step it is recomputed as a weighted combination of all encoder hidden states (h) considering their relevance to the current decoder state (d):

$$c_t = \sum_{i=1}^T a(d_{t-1}, h_i) h_i$$

The weights measure how relevant each position of the source (i) is for generating the token at the current positions of the target (t). The score that produce the weights can be any of the scoring functions we have discussed, where the query is the decoder's previous hidden state and keys and values are the encoder's hidden states. For example, we can use the additive score to compute the attention weights:

$$s(d_{t-1}, h_i) = w^T \tanh(W_q d_{t-1} + W_k h_i + b)$$

Then the attention weights are obtained by applying softmax to the scores:

$$a(d_{t-1}, h_i) = \frac{\exp(s(d_{t-1}, h_i))}{\sum_{j=1}^T \exp(s(d_{t-1}, h_j))}$$

Our current implementation of the encoder already returns both the output and the state, so it can be used as it is. The decoder, however, needs to be modified to compute attention at each step. We can implement this by adding an attention module that takes the decoder's previous hidden state and the encoder's outputs to compute the context vector, which is then concatenated with the embedded input token before being fed into the GRU:

```
class AttentionDecoder(nn.Module):

    def __init__(self, embedding, hidden_dim, num_layers, dropout=0.0):
        super().__init__()

        # Store the embedding layer to convert token IDs into dense vectors
        self.embedding = embedding

        # Create the attention layer to dynamically attend to encoder hidden states
        self.attention = AdditiveAttention(d_q=hidden_dim,
                                           d_k=hidden_dim,
                                           h=hidden_dim)

        # Create a multi-layer GRU that takes the concatenation of the embedding
        # and the context vector as input
        self.rnn = nn.GRU(embed_dim + hidden_dim, hidden_dim, num_layers,
                           batch_first=True,
                           dropout=dropout if num_layers > 1 else 0)

        # Create a linear layer to project the hidden states to the vocabulary size
        # for output prediction
        self.fc = nn.Linear(hidden_dim, ita_vocab_size)

    def forward(self, Y, state, enc_output):

        # Embed the input sequence of token IDs into dense vectors
        emb = self.embedding(Y)

        outputs = []

        for t in range(emb.shape[1]):

            # Use the last decoder hidden state as the query to attend over
            # the encoder outputs (keys and values)
            query = state[-1].unsqueeze(1)
            context = self.attention(query, enc_output, enc_output)
```

```
        # Concatenate the current token embedding with the context vector
        # and pass through the GRU
        output, state = self.rnn(torch.cat([emb[:, t:t+1, :], context], dim=-1),
    ↪ state)
        outputs.append(output)

        # Project the concatenated outputs onto the vocabulary size to get logits
        logits = self.fc(torch.cat(outputs, dim=1))

    return logits, state
```

1.6.2 Bahdanau model

We now wrap encoder and decoder into a single module, mirroring the structure of Seq2Seq, but with one key difference: instead of passing only the final encoder hidden state as a fixed context vector, the full sequence of encoder hidden states is forwarded to the decoder at every step. This allows the attention mechanism introduced by Bahdanau et al. (2015) to dynamically recompute a different context vector at each decoding step, selectively focusing on the most relevant parts of the source sequence rather than being forced to retrieve everything from a single compressed representation.

```
class AttentionSeq2Seq(nn.Module):

    def __init__(self, encoder, decoder):
        super().__init__()

        # Store the encoder
        self.encoder = encoder

        # Store the decoder
        self.decoder = decoder

    def forward(self, X, Y):

        # Encode the source sequence and get the encoder outputs and final state
        enc_output, state = self.encoder(X)

        # Decode the target sequence using the encoder outputs for attention
        logits, _ = self.decoder(Y, state, enc_output)
```

```
# return the logits
return logits
```

We instantiate the attention model with the same hyperparameters used for the baseline, so that any difference in performance can be attributed to the attention mechanism rather than to a larger or different architecture:

```
# Instantiate the attention-based encoder
attention_encoder = Encoder(eng_embedding, hidden_dim=512, num_layers=2, dropout=0.3)

# Instantiate the attention-based decoder
attention_decoder = AttentionDecoder(ita_embedding, hidden_dim=512, num_layers=2,
    ↪ dropout=0.3)

# Instantiate the attention-based seq2seq model
attention_model = AttentionSeq2Seq(attention_encoder, attention_decoder).to(device)

# Create an optimizer for the attention-based seq2seq model parameters
attention_optimizer = torch.optim.Adam(attention_model.parameters(), lr=1e-3)
```

We check the number of trainable parameters to confirm that the attention model is indeed more expressive than the baseline:

```
# Compute the total number of trainable parameters in the attention-based seq2seq
    ↪ model
total_params = sum(p.numel() for p in attention_model.parameters() if p.requires_grad)

# Print the total number of trainable parameters
print(f"Attention model parameters: {total_params:,}")
```

Attention model parameters: 26,319,810

1.6.3 Training

We train the attention model for some epochs using the same procedure as the baseline, caching the weights and loss curves to disk so that subsequent runs skip training and reload the checkpoint directly:

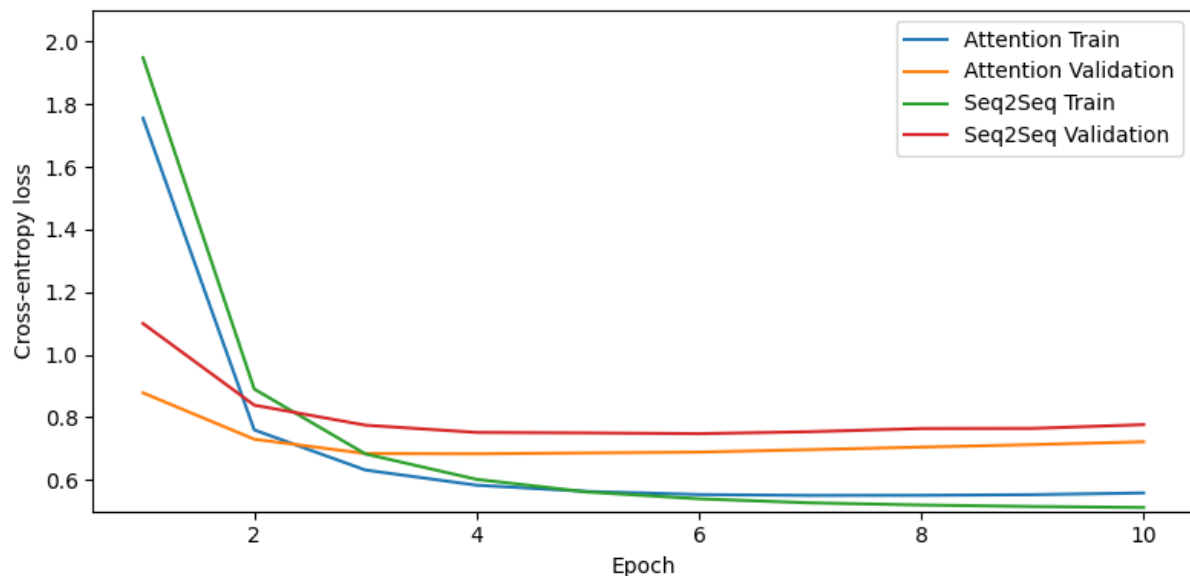
```
# Define the path to the model checkpoint file
model_path = "./data/attention_model.pt"

# Train the attention-based model and obtain the training and validation losses for
↳ each epoch
attention_train_losses, attention_val_losses = train_model(attention_model,
↳ attention_optimizer, model_path)
```

Epoch 10 | train loss 0.5585 | val loss 0.7221
Model saved to ./data/attention_model.pt

We plot the training and validation loss curves also for the attention model to visually compare the training dynamics with the baseline:

```
plt.figure(figsize=(8, 4))
plt.plot(range(1, num_epochs + 1), attention_train_losses, label='Attention Train')
plt.plot(range(1, num_epochs + 1), attention_val_losses, label='Attention
↳ Validation')
plt.plot(range(1, num_epochs + 1), seq2seq_train_losses, label='Seq2Seq Train')
plt.plot(range(1, num_epochs + 1), seq2seq_val_losses, label='Seq2Seq Validation')
plt.xlabel('Epoch')
plt.ylabel('Cross-entropy loss')
plt.ylim(0.5, 2.1)
plt.legend()
plt.tight_layout()
plt.show()
```



Both models converge quickly within the first few epochs and then plateau. The baseline shows a wider and growing gap between train and validation loss, a sign of mild overfitting. The attention model keeps the two curves closer together for longer, suggesting that **the additional context acts as a regulariser**.

1.6.4 Performance

The prediction function mirrors the one used for the baseline model with two additions: the full sequence of encoder hidden states is passed to the decoder at every step so that attention can dynamically compute a context vector, and after each decoding step the attention weights are collected and returned as a matrix, where each row shows which source tokens the model focused on when generating the corresponding target token:

```
def attention_predict(source, max_len=20):

    # Set the model to evaluation mode
    attention_model.eval()

    # Disable gradient computation since we are only performing inference
    with torch.no_grad():

        # Get the indices for the padding, beginning-of-sequence, and unknown tokens
        pad = eng_to_idx['<pad>']
        bos = ita_to_idx['<bos>']
```

```
unk = eng_to_idx['<unk>']

# Wrap the source sentence with <bos> and <eos> to match training format
source = ['<bos>'] + source + ['<eos>']

# Store the length of the source sentence
src_len = len(source)

# Encode the source sentence
src_ids = [eng_to_idx.get(token, unk) for token in source][:max_len]

# Pad the source sentence with the padding token
src_ids += [pad] * (max_len - len(src_ids))

# Convert the list of source token ids into a tensor and move it to the
↳ selected device
src = torch.tensor(src_ids, dtype=torch.long).unsqueeze(0).to(device)

# Run the encoder on the source sentence
enc_output, context = attention_model.encoder(src)

# Initialize the decoder input with the beginning-of-sequence token
# and the list to store the predicted output tokens and attention weights
token, output, all_weights = torch.tensor([[bos]],
↳ dtype=torch.long).to(device), [], []

# Iteratively generate tokens until the end-of-sequence token is produced
# or the maximum length is reached
for _ in range(max_len):

    # Perform a forward pass through the decoder using the current token,
    # context, and encoder outputs
    logits, context = attention_model.decoder(token, context, enc_output)

    # Select the token with the highest predicted probability as the next token
    token = logits.argmax(dim=-1)

    # Convert the predicted token index back to a word using the Italian
    ↳ vocabulary
    word = idx_to_ita[token.item()]

    # If the predicted word is the end-of-sequence token, stop generating
    ↳ further tokens
```

```

        if word == '<eos>':
            break

        # Append the predicted word to the output list
        output.append(word)

        # Append the attention weights from the decoder's attention mechanism to
        ↪ the list
        # of all weights for later analysis
        ↪ all_weights.append(attention_model.decoder.attention.weights.squeeze().cpu())

        # Stack weights and slice out <bos> and <eos> columns
        weights = torch.stack(all_weights, dim=0)[: , 1:src_len+1]

        # Return the list of predicted output tokens
        # and the tensor of attention weights for all generated tokens
        return output, weights

```

We evaluate the model on the validation set using the same BLEU metric computed earlier, so we can directly compare translation quality before and after introducing attention:

```

# Evaluate the BLEU score of the attention-based model on the test set
attention_bleu_overall, attention_bleu_by_length = evaluate_bleu(attention_model,
                                                                ↪ lambda src: attention_predict(src)[0])

# Print the overall BLEU score for both models to compare their performance
print((f"Seq2Seq Overall BLEU: {seq2seq_bleu_overall:.4f}"))
print((f"Attention Overall BLEU: {attention_bleu_overall:.4f}"))

```

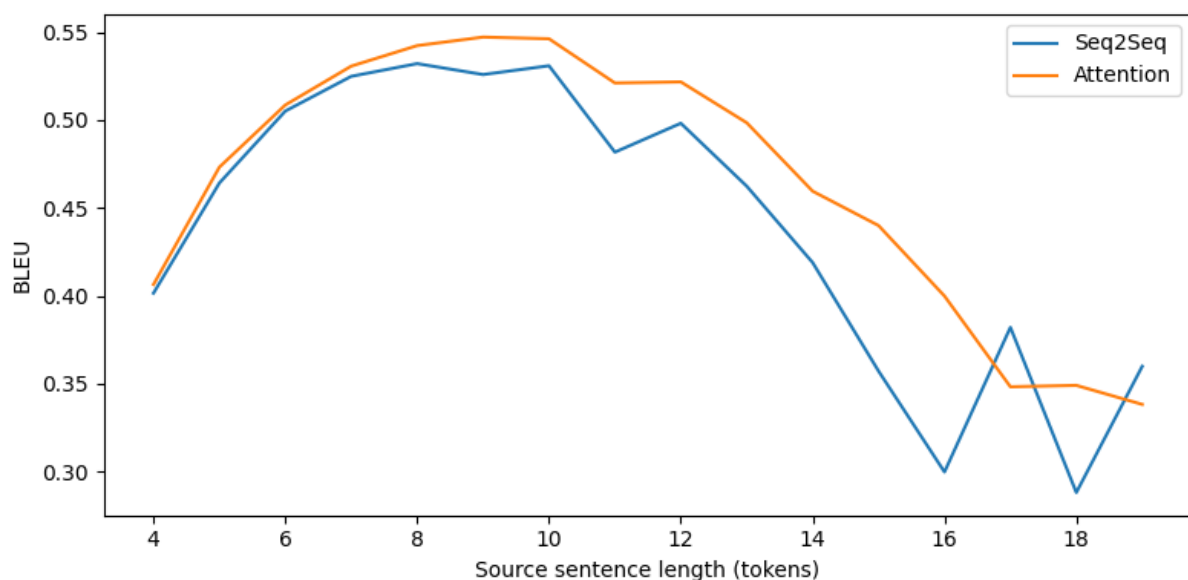
Seq2Seq Overall BLEU: 0.4960

Attention Overall BLEU: 0.5072

The attention model scores marginally higher, but the difference is negligible. This is not surprising: the dataset consists mostly of short sentences where a fixed context vector is already sufficient, so the dynamic has little room to show its advantage. Attention becomes significantly more impactful on longer, more complex sequences. To understand where the attention mechanism helps most, we break down the BLEU score by source sentence length:

```
# Extract the BLEU scores for the selected lengths
attention_bleus = [attention_bleu_by_length[l] for l in lengths]

# Create a line plot of BLEU scores by source sentence length
plt.figure(figsize=(8, 4))
plt.plot(lengths, seq2seq_bleus, label='Seq2Seq')
plt.plot(lengths, attention_bleus, label='Attention')
plt.xlabel('Source sentence length (tokens)')
plt.ylabel('BLEU')
plt.legend()
plt.tight_layout()
plt.show()
```



The attention model consistently outperforms the baseline across all lengths, with the gap widening as sentences grow longer, exactly where the fixed context vector bottleneck hurts most and the ability to dynamically focus on relevant source tokens provides the greatest advantage.

1.6.5 Showing attention

Beyond aggregate metrics, we can inspect a few hand-picked sentences to get a qualitative feel for where attention helps and where both models still struggle:

```
# For each previously defined example sentence, compare translations using both
# the base seq2seq model and the attention-based model
for src in examples:
    base = seq_2_seq_predict(src)
    attention, _ = attention_predict(src)
    print(f" EN:          {' '.join(src)}")
    print(f" Seq2Seq:    {' '.join(base)}")
    print(f" Attention: {' '.join(attention)}")
    print()
```

EN: i love you .
Seq2Seq: io vi amo .
Attention: ti amo .

EN: the cat is on the table .
Seq2Seq: il gatto è sdraiato sul tavolo .
Attention: il gatto è sul tavolo .

EN: where is the train station ?
Seq2Seq: dov ' è la stazione ferroviaria ?
Attention: dov ' è la stazione dei treni ?

EN: good morning !
Seq2Seq: buongiorno !
Attention: buongiorno !

EN: she reads a book every day .
Seq2Seq: lei legge un libro ogni giorno .
Attention: legge un libro ogni giorno .

EN: we are happy .
Seq2Seq: siamo felici .
Attention: siamo felici .

EN: what time is it ?
Seq2Seq: che ora è ?
Attention: che ore è ?

EN: can you help me ?

Seq2Seq: potete aiutarmi ?

Attention: puoi aiutarmi ?

EN: this is a test .

Seq2Seq: questo è un esame .

Attention: questo è un test .

Both models handle short, frequent sentences equally well. Sentences like "i love you", "the cat is on the table", and "where is the train station" produce identical and correct translations. The differences emerge on slightly harder cases: for "good morning" the baseline produces the literal "buona mattina" while attention generates the more natural "buongiorno", and for "she reads a book every day" the baseline hallucinates "sulla giornata" (on the day) where attention correctly produces "ogni giorno" (every day). The subject pronoun drop in "siamo felici" rather than "noi siamo felici" is grammatically correct in Italian, not an error. Overall the qualitative improvement is modest, consistent with what the BLEU scores showed: attention helps at the margin but the task remains well within reach of the baseline for short, frequent sentences. To further illustrate how attention helps on longer and more complex sentences, we test both models on a sentence with a repeated pronoun ("she ... she") that requires tracking coreference across several tokens:

```
src = ["she", "told", "me", "that", "she", "would", "come", "to", "the", "party",  
      ↪ "tomorrow", "."]  
  
seq2seq_translation = seq_2_seq_predict(src)  
attention_translation, attention_weights = attention_predict(src)  
  
print(f" EN:          {' '.join(src)}")  
print(f" Seq2Seq:      {' '.join(seq2seq_translation)}")  
print(f" Attention: {' '.join(attention_translation)}")
```

EN: she told me that she would come to the party tomorrow .

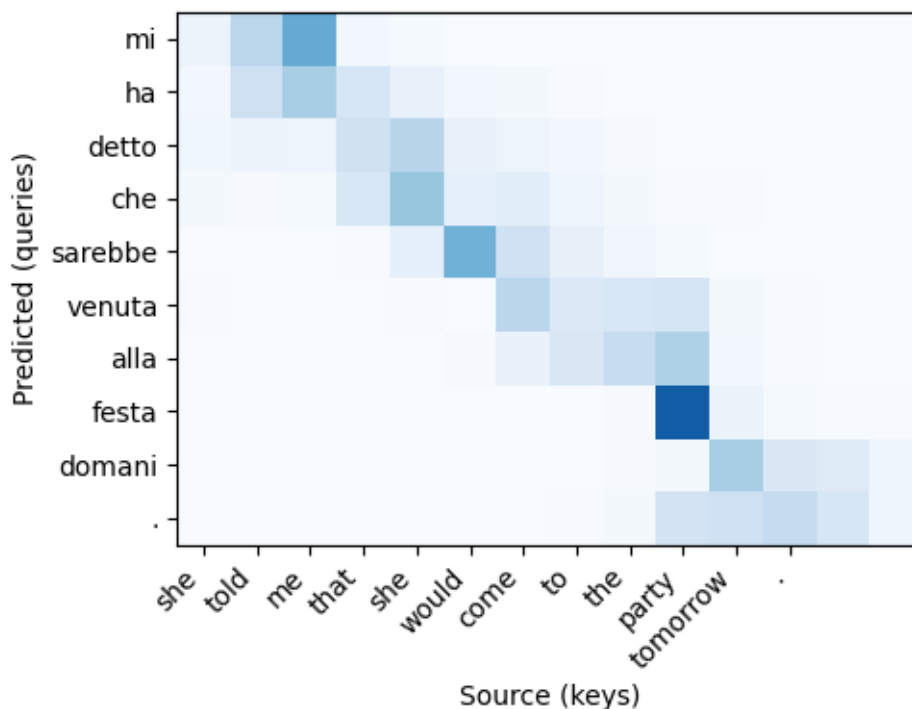
Seq2Seq: mi ha detto che sarebbe andato a venire alla festa domani .

Attention: mi ha detto che sarebbe venuta alla festa domani .

The attention model produces a noticeably better translation. The baseline correctly captures the beginning ("mi ha detto che") but then loses track of the subject: "lei era" (she was) instead

of "sarebbe venuta" (she would come), missing the conditional mood entirely and hallucinating "di domani sera" (of tomorrow evening) instead of simply "domani" (tomorrow). The attention model handles both correctly, suggesting it successfully attends to "would come" and "tomorrow" independently when generating the corresponding Italian tokens. We can visualise the attention weight matrix to inspect which source tokens the decoder focused on at each decoding step:

```
show_heatmap(attention_weights,
              xlabel="Source (keys)", ylabel="Predicted (queries)",
              xticks=src,
              yticks=attention_translation)
```



The heatmap reveals a near-diagonal alignment pattern: each Italian token attends most strongly to its English counterpart. For content words *sarebbe*/*would*, *venuta*/*come*, *festa*/*party*, *domani*/*tomorrow* the peak is sharp and well-placed. Function words like *mi* and *che* show slightly broader attention, reflecting structural differences between the two languages. This emergent alignment was **never explicitly supervised**: the model learned it purely from translation examples.

1.7 The transformer architecture

The RNN encoder–decoder with attention is a significant step forward, but it still carries one **fundamental limitation** inherited from the recurrent architecture: **tokens are processed sequentially**. At each time step, the GRU must wait for the previous hidden state before it can compute the next one, which means the computation cannot be parallelized across the sequence dimension. This has two practical consequences: **training is slow** on long sequences, and **gradients must travel through many recurrent steps** to reach early tokens, making it harder to capture long-range dependencies even when attention is present. The natural question is: if the decoder already uses attention to dynamically query the encoder at every step, **do we actually need the recurrence at all?** Vaswani et al. (2017) in *Attention Is All You Need* answered with a definitive no, replacing the GRU entirely with attention, and processing all tokens in parallel in a single forward pass.

1.7.1 Self-attention

The GRU's role in the encoder is to build a representation of each token that is informed by its context, the tokens that surround it. But this is precisely what attention already does: it aggregates information from a set of positions weighted by relevance. If we apply attention not between two different sequences but **within the same sequence** (letting each position query all other positions) we can build those context-aware representations without any recurrence at all. This is the idea behind **self-attention**. Given a sequence of vectors, each position produces its own query, key, and value through three separate learned linear projections. Each position then attends to every other position in the same sequence, producing an output that is a weighted combination of all values, where the weights reflect how relevant each position is to the current one. Crucially, all positions are processed **simultaneously**: there is no sequential dependency, so the entire sequence can be handled in a single parallel operation, directly addressing the bottleneck identified above. Given an input sequence

$$X \in \mathbb{R}^{N \times D}$$

queries, keys and values are obtained through learned linear projections:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

where

$$W_Q, W_K, W_V \in \mathbb{R}^{D \times h}$$

are learnable weight matrices projecting each token into a h -dimensional space. The matrices are independent and learned separately, so the model can discover different aspects of each token depending on whether it is acting as a query, a key, or a value. The projected dimension

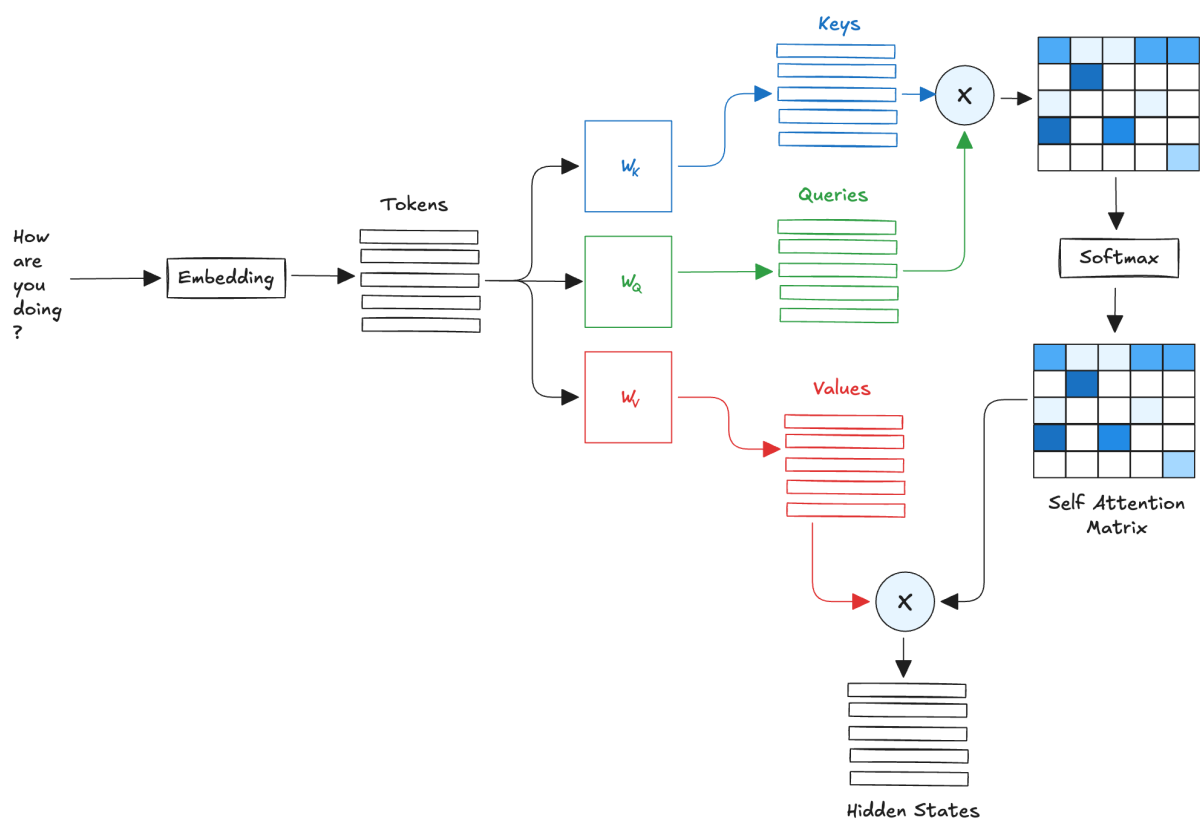
(h) is typically chosen smaller than the input dimension (D), acting as a bottleneck that keeps the computation efficient. Self-attention is then computed as:

$$\text{SelfAttention}(X) = \text{softmax}\left(\frac{XW_Q(XW_K)^T}{\sqrt{h}}\right)XW_V$$

The matrix:

$$\frac{XW_Q(XW_K)^T}{\sqrt{h}}$$

collects the **compatibility score between every pair of positions**, and the softmax is applied row-wise so that each position's attention weights sum to one.



```
class SelfAttention(nn.Module):

    def __init__(self, embed_dim, h):
        super().__init__()

        # Linear projections for queries, keys, and values
        self.W_q = nn.Linear(embed_dim, h, bias=False)
        self.W_k = nn.Linear(embed_dim, h, bias=False)
```

```

self.W_v = nn.Linear(embed_dim, h, bias=False)

def forward(self, X):

    # Project the input sequence into queries, keys, and values
    Q = self.W_q(X)
    K = self.W_k(X)
    V = self.W_v(X)

    # Compute scaled dot-product attention scores
    output, self.weights = scaled_dot_product_attention(Q, K, V)

    return output

```

1.7.2 Multi-head attention

A single self-attention operation computes one weighted combination of values for each position, guided by a single set of learned projections. This means the model is **limited** to capturing **one type of relationship at a time**, for example syntactic agreement or semantic similarity, but not both simultaneously. Multi-head attention addresses this by running **several independent self-attention operations in parallel**, each with its own set of projection matrices. Each of these parallel operations is called a **head**, and because each head learns its own notion of query, key, and value, different heads are free to specialize in capturing different types of relationships between positions. The outputs of all heads are then concatenated and projected back to the original dimension, so the overall shape of the computation remains the same regardless of how many heads are used. Given H heads, each head maintains its own projection matrices:

$$W_Q^{(i)}, W_K^{(i)}, W_V^{(i)} \in \mathbb{R}^{D \times d}$$

where

$$d = D/H$$

so the each head works in a lower-dimensional space, so the total computational cost is roughly the same as a single full-dimensional attention operation.

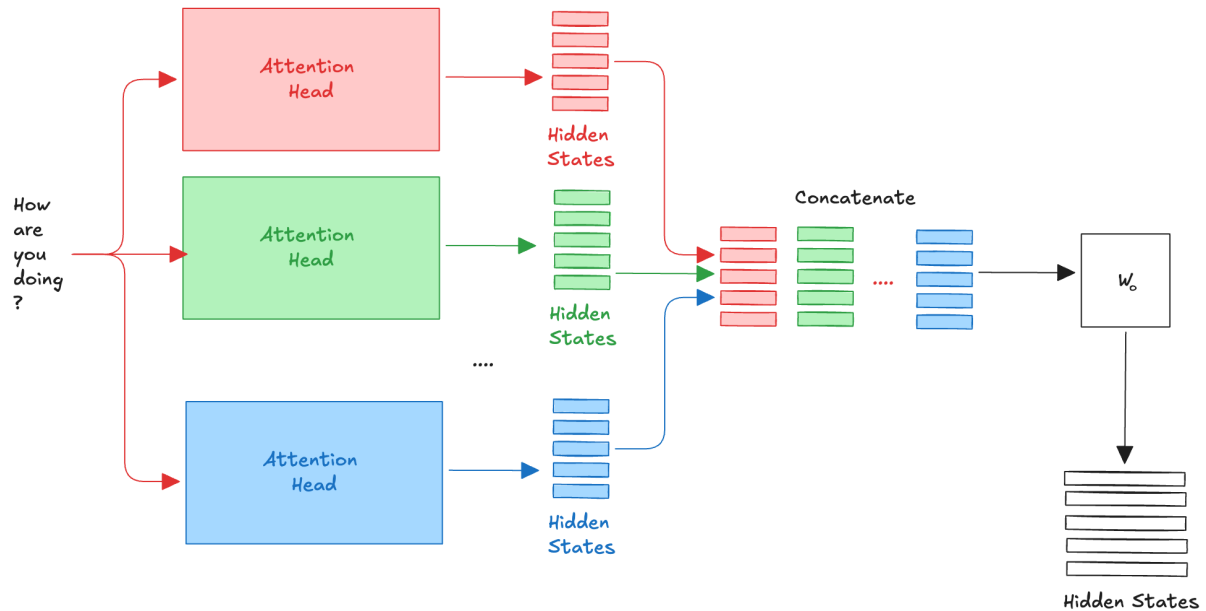
and computes its own self-attention output:

$$\text{head}_i(X) = \text{softmax}\left(\frac{XW_Q^{(i)}(XW_K^{(i)})^\top}{\sqrt{d}}\right)XW_V^{(i)} \in \mathbb{R}^{N \times d}$$

The outputs are then concatenated along the feature dimension and projected back through a final learned matrix:

$$W_O \in \mathbb{R}^{D \times D}$$

$$\text{MultiHead}(X) = [\text{head}_1(X) \parallel \text{head}_2(X) \parallel \dots \parallel \text{head}_H(X)] W_O$$



```
class MultiHeadAttention(nn.Module):

    def __init__(self, embed_dim, num_heads):
        super().__init__()

        # Store the number of attention heads and the dimension of each head
        self.num_heads = num_heads
        self.d = embed_dim // num_heads

        # Linear projections for queries, keys, values, and output
        self.W_q = nn.Linear(embed_dim, embed_dim, bias=False)
        self.W_k = nn.Linear(embed_dim, embed_dim, bias=False)
        self.W_v = nn.Linear(embed_dim, embed_dim, bias=False)
        self.W_o = nn.Linear(embed_dim, embed_dim, bias=False)

    def forward(self, X):

        # Get the batch size, sequence length, and embedding dimension from the input
        batch, N, D = X.shape

        # Project and split into H heads: (batch, N, H, d) → (batch, H, N, d)
        Q = self.W_q(X).view(batch, N, self.num_heads, self.d).transpose(1, 2)
```

```
K = self.W_k(X).view(batch, N, self.num_heads, self.d).transpose(1, 2)
V = self.W_v(X).view(batch, N, self.num_heads, self.d).transpose(1, 2)

# Scaled dot-product attention for all heads in parallel
self.weights, out = scaled_dot_product_attention(Q, K, V)

# Concatenate heads and project back: (batch, N, D)
out = out.transpose(1, 2).contiguous().view(batch, N, D)

# Final linear projection to combine the outputs of all heads
return self.W_o(out)
```

The key implementation trick is to avoid a Python loop over heads: instead, a single large projection of size D is reshaped into H chunks of size d , so all heads are computed simultaneously via a single batched matrix multiplication.

1.7.3 Positional encoding

Self-attention is an extremely powerful mechanism, but it comes with an important limitation: it has **no intrinsic notion of order**. A self-attention layer treats its input essentially as a set of tokens rather than as an ordered sequence. For each position, it computes a weighted combination of all other positions, but the operation itself is completely symmetric. If we shuffle the tokens in the input sentence, the outputs are shuffled in exactly the same way. The model can see which tokens are present, but not where they appear. In other words, **self-attention alone has no built-in concept of "before" and "after"**. This is very different from recurrent architectures, in which order is implicitly encoded by the computation itself: the hidden state at time t is produced by repeatedly updating a memory that depends on all previous steps. The sequence structure is therefore baked directly into the recurrence. To fix this problem, we can **explicitly inject positional information into the token representations**. Before the embeddings are passed into the attention layers, a position-dependent vector is added to each token embedding. The resulting representation therefore contains two complementary pieces of information: **what the token is**, provided by the embedding, and **where the token occurs** in the sequence, provided by the positional encoding. There are many ways to encode positions. At first glance, it seems natural to use a simple scalar index

0, 1, 2, 3, ...

attached directly to the embedding. However, sequence data contains structure at many scales simultaneously. In language for example we have local word order, phrase structure, sentence structure, paragraph-level dependencies. A single scalar cannot capture all of these at once. The

original Transformer introduced a particularly elegant solution based on **sinusoidal functions**. Rather than representing a position as a single number, each position is mapped to a vector of dimension D using sine and cosine waves at **geometrically spaced frequencies**. For a sequence position

$$p \in \{0, \dots, N - 1\}$$

and a dimension index

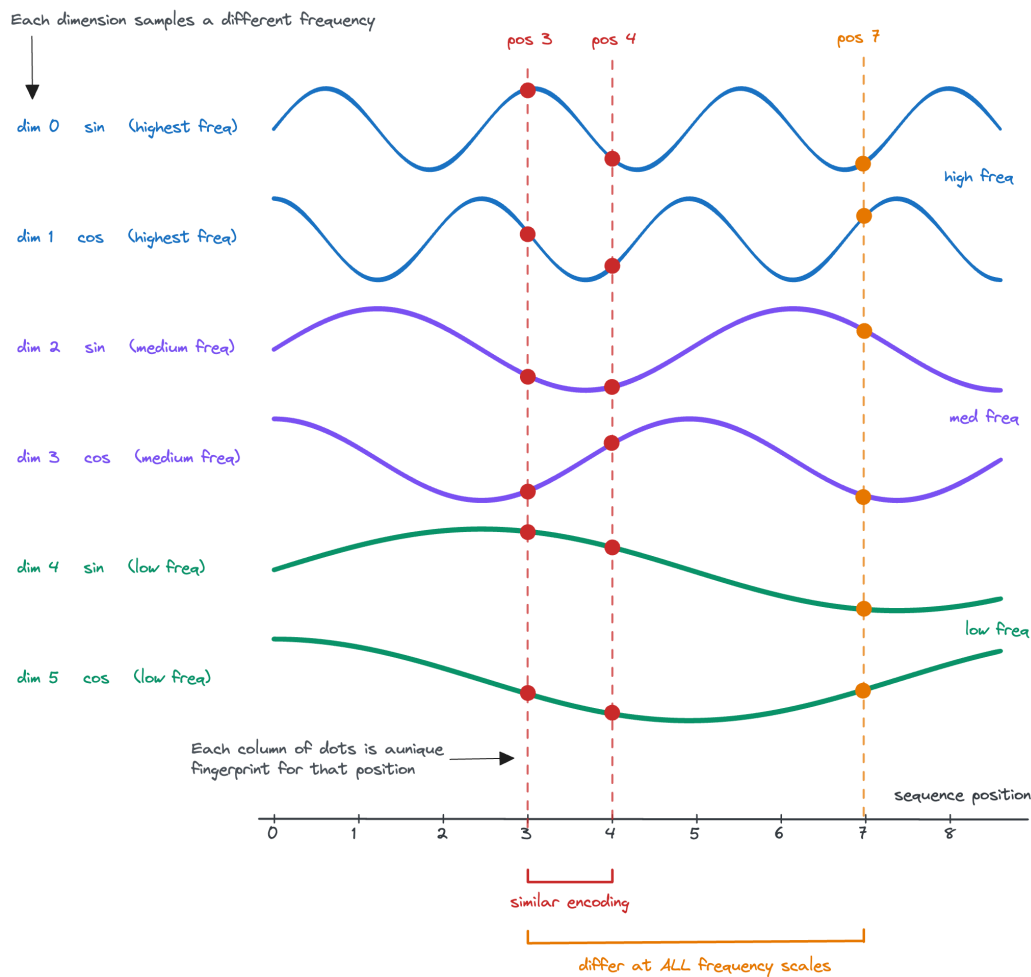
$$i \in \{0, \dots, D/2 - 1\}$$

the **sinusoidal positional encoding** is defined as:

$$\text{PE}(p, 2i) = \sin\left(\frac{p}{10000^{2i/D}}\right)$$

$$\text{PE}(p, 2i + 1) = \cos\left(\frac{p}{10000^{2i/D}}\right)$$

The key insight is that the frequency changes with i : low-index dimensions oscillate rapidly, high-index dimensions oscillate slowly. This means that **"nearness" between two positions is not a single number, it is a profile that looks different across dimensions**. Two positions that are close together (say, position 3 and position 4) will produce very similar values in the slow, low-frequency dimensions, but already differ noticeably in the fast, high-frequency ones. Two positions that are far apart (say, position 3 and position 40) will disagree everywhere, at every frequency scale. It is a bit like reading a clock: the hour hand tells coarse time, the minute hand tells finer time, the second hand finer still. You need all three together to know exactly where we are, and two moments that are close will agree on the hours but differ on the seconds. This multi-scale structure creates a **unique geometric signature** for every position in the sequence, as illustrated below:



It also has a practical consequence: self-attention, which operates through dot products between vectors, naturally learns to compare these signatures and can therefore reason about both **absolute positions** (where a token sits in the sequence) and **relative distances** (how far apart two tokens are). Without positional encoding, the sentence

"the cat chased the cat"

would present two identical representations of "cat" to the attention mechanism, with no way to tell them apart. The positional encoding breaks that symmetry by injecting location directly into the representation:

$$\tilde{x}_p = x_p + \text{PE}(p, :)$$

An elegant property of this construction is that it introduces **no additional learned parameters**: the encoding vectors are generated deterministically from the formulas above and reused at every forward pass. Despite this simplicity, the resulting positional structure is rich enough for Transformers to reason effectively about sequence order in sequences of arbitrary length.

```
class PositionalEncoding(nn.Module):

    def __init__(self, embed_dim, max_len=1000):
        super().__init__()

        # Allocate the positional encoding matrix: shape (max_len, embed_dim)
        PE = torch.zeros(max_len, embed_dim)

        # Column vector of positions: shape (max_len, 1)
        positions = torch.arange(max_len).unsqueeze(1)

        # Divisor term: one value per pair of dimensions
        # shape (embed_dim // 2,)
        div_term = 10000 ** (torch.arange(0, embed_dim, 2) / embed_dim)

        # Even dimensions get sine, odd dimensions get cosine
        PE[:, 0::2] = torch.sin(positions / div_term)
        PE[:, 1::2] = torch.cos(positions / div_term)

        # Register as a buffer: it moves with the model (e.g. to GPU)
        # but is not treated as a learnable parameter
        self.register_buffer('PE', PE)

    def forward(self, X):
        # Add the positional encoding for the first seq_len positions
        return X + self.PE[:X.size(1)]
```

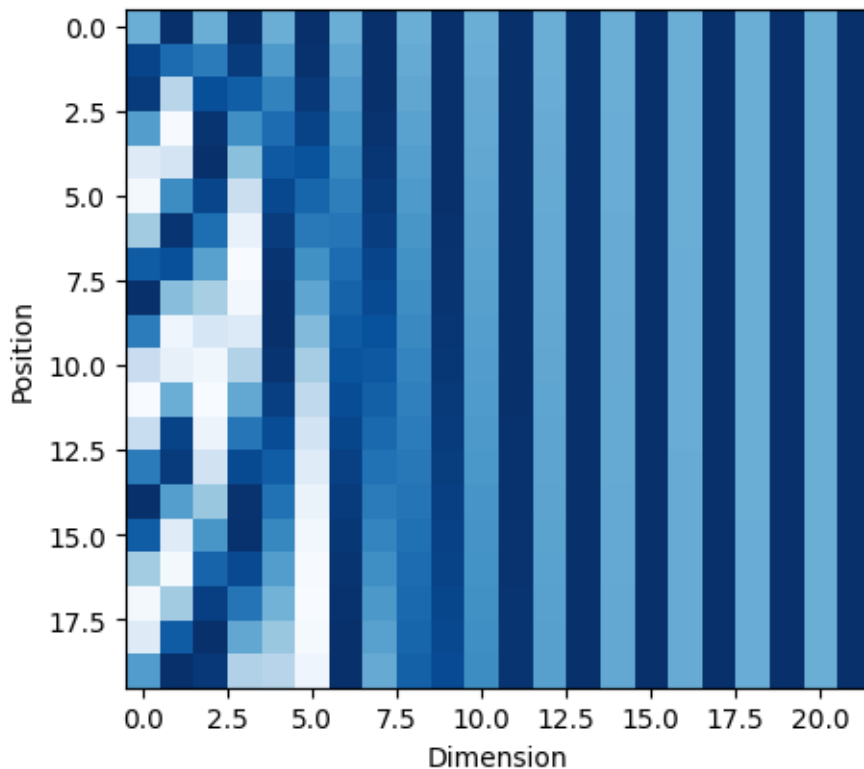
To build an intuition for the structure of the encoding, we can visualise the PE matrix as a heatmap. Each row corresponds to one position in the sequence, and each column corresponds to one dimension of the encoding. We use a small embedding dimension of 22 so that the alternating sine/cosine pattern across pairs of dimensions is clearly visible.

```
# Instantiate a positional encoding for an embedding dimension
pos_enc = PositionalEncoding(embed_dim=22, max_len=20)

# Extract the encoding matrix for the first 20 positions
# and clamp it to [0, 1] just for display purposes
PE_matrix = (pos_enc.PE[:20] + 1) / 2

# Show as a heatmap: rows are positions, columns are dimensions
show_heatmap(PE_matrix,
              xlabel='Dimension',
```

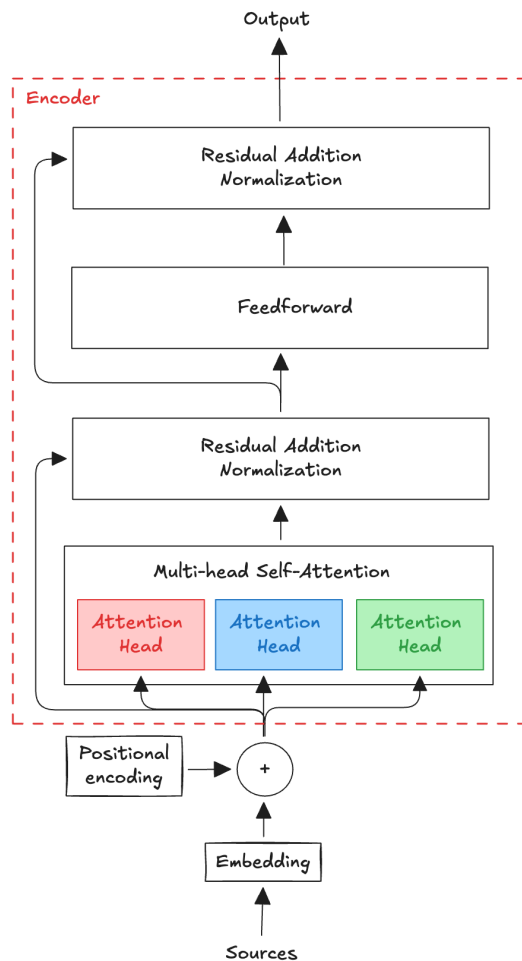
```
ylabel='Position')
```



The leftmost columns (dim 0–4) flicker rapidly between light and dark as we scan down the rows, those are the high-frequency dimensions, where even adjacent positions already differ. Moving right, the oscillation visibly slows: by dim 6–8 we can see broad bands spanning several rows rather than individual cells. By the time we reach dim 10 onward, the columns are almost uniform, those low-frequency dimensions barely move across just 20 positions, so they carry coarse, global location information rather than fine-grained local detail. We can also see the sine/cosine pairing at work: within each frequency pair, adjacent columns tend to be out of phase with each other, one dark while the other is light, which is exactly what you’d expect from a sin/cos pair sharing the same frequency. Finally, notice that every row is distinct: no two positions share the same pattern of light and dark across all 22 dimensions. That is the unique geometric signature the formula is designed to guarantee. And if we compare two nearby rows (say position 3 and position 4) they look almost identical in the right half of the matrix and only start to diverge in the left few columns. Compare position 0 with position 19 and they differ almost everywhere. Modern models such as GPT replace the fixed sinusoidal formula with a **learned positional embedding table**, treating positions the same way tokens are treated. The table is initialised randomly and updated alongside all other parameters. The two approaches perform similarly in practice.

1.7.4 Encoder block

All the ingredients are now in place: token embeddings, positional encoding, and multi-head self-attention. What remains is to wire them into a **single repeatable unit** that can be stacked to form the full encoder. This unit is called an **encoder block**, and the original Transformer paper defines it as a fixed arrangement of four components:



After each major transformation, the model adds the original input representation back to the transformed output through a **residual connection**. This means that a layer is not forced to completely rewrite the representation it receives. Instead, it only needs to learn how the current representation should be adjusted or refined. This turns the learning problem into something much easier: rather than constructing an entirely new representation from scratch, the network learns incremental corrections to an already meaningful signal. This simple addition has another extremely important effect. As Transformers become deeper, gradients must propagate backward through many layers during training. Without residual connections, the signal can progres-

sively weaken or become unstable, making optimization difficult. Residual connections create a direct route through which information and gradients can flow, allowing very deep models to remain trainable. Immediately after the residual addition, the representation is stabilized using **layer normalization**. The role of this operation is to keep activations within a controlled numerical range so that successive layers operate on signals with consistent scale. Self-attention itself is primarily a mechanism for exchanging information across the sequence. Each token gathers relevant information from other positions and combines it through weighted averaging. This makes attention extremely effective at routing and aggregating contextual information, but the operation remains largely linear in nature. By itself, attention mainly redistributes existing information rather than deeply transforming it. The nonlinear processing inside the encoder block comes from the **feed-forward network** that follows the attention layer. This network is applied independently to every token position and acts like a small neural network operating on each contextualized token representation. A useful way to think about the architecture is that attention decides which information should be collected, while the feed-forward network decides how that information should be interpreted and transformed. Together, residual connections, layer normalization, self-attention, and feed-forward processing form the fundamental computational unit of the Transformer encoder:

```
class EncoderBlock(nn.Module):

    def __init__(self, embed_dim, num_heads, ffn_dim, dropout=0.0):
        super().__init__()

        # Multi-head self-attention
        self.attention = MultiHeadAttention(embed_dim, num_heads)

        # Feed-forward network
        self.linear1 = nn.Linear(embed_dim, ffn_dim)
        self.linear2 = nn.Linear(ffn_dim, embed_dim)

        # Layer normalisation
        self.norm1 = nn.LayerNorm(embed_dim)
        self.norm2 = nn.LayerNorm(embed_dim)

        # Dropout
        self.dropout = nn.Dropout(dropout)

    def forward(self, X):

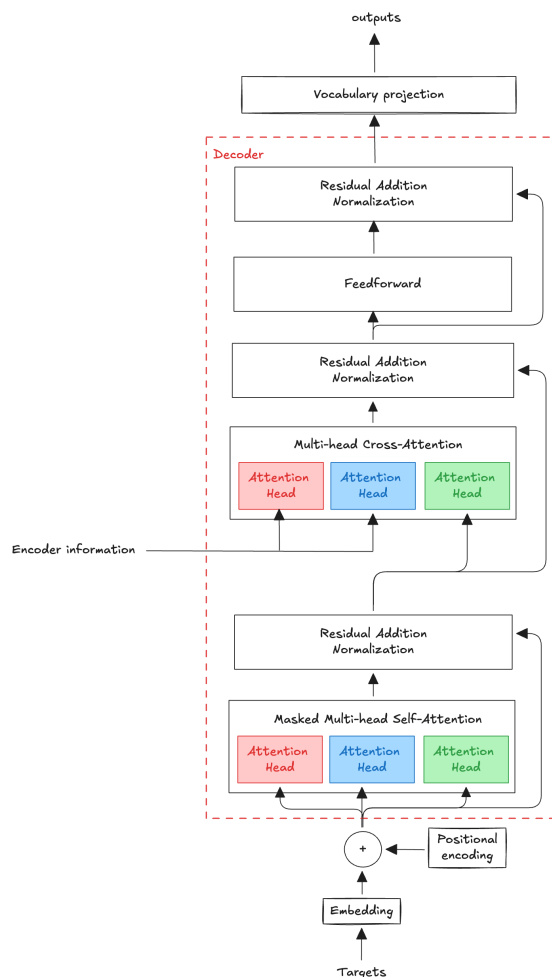
        # Self-attention: attend → dropout → residual → normalise
        X = self.norm1(X + self.dropout(self.attention(X)))
```

```
# Feed-forward: expand → activate → project → dropout → residual → normalise
ffn_out = self.linear2(F.relu(self.linear1(X)))
X = self.norm2(X + self.dropout(ffn_out))

return X
```

1.7.5 Decoder block

The decoder block is structurally similar to the encoder, but it has two additional responsibilities. First, it must generate tokens one at a time without being allowed to see future positions in the target sequence. Second, at each generation step it must incorporate information from the encoder output to condition the translation on the source sentence:



During training, the full target sequence is available at once, which is efficient but creates a risk:

without any constraint, position i would be free to attend to future positions, which are tokens that have not been generated yet. This would allow the model to “cheat” by reading the answer before producing it. To prevent this, a **causal mask** is added to the attention scores before the softmax, so future positions are effectively invisible to the model:

```
class MaskedMultiHeadAttention(nn.Module):

    def __init__(self, embed_dim, num_heads):
        super().__init__()

        self.num_heads = num_heads
        self.d = embed_dim // num_heads

        self.W_q = nn.Linear(embed_dim, embed_dim, bias=False)
        self.W_k = nn.Linear(embed_dim, embed_dim, bias=False)
        self.W_v = nn.Linear(embed_dim, embed_dim, bias=False)
        self.W_o = nn.Linear(embed_dim, embed_dim, bias=False)

    def forward(self, X):

        B, N, _ = X.shape

        Q = self.W_q(X).view(B, N, self.num_heads, self.d).transpose(1, 2)
        K = self.W_k(X).view(B, N, self.num_heads, self.d).transpose(1, 2)
        V = self.W_v(X).view(B, N, self.num_heads, self.d).transpose(1, 2)

        mask = torch.triu(torch.full((N, N), float('-inf')), device=X.device),
        ↪ diagonal=1)
        scores = Q @ K.transpose(-2, -1) / (self.d ** 0.5) + mask
        weights = F.softmax(scores, dim=-1)
        out = weights @ V

        out = out.transpose(1, 2).contiguous().view(B, N, self.num_heads * self.d)
        return self.W_o(out)
```

The **Cross-attention** layer ensures that queries come from the decoder, keys and values come from the encoder output, allowing the decoder to selectively read the source sentence at each step. Finally, the feed-forward network provides the same nonlinear processing as in the encoder, but now it operates on representations that have been enriched both by self-attention and by cross-attention, so it can learn to interpret the combined information from both the source and the target context.

```
class CrossAttention(nn.Module):

    def __init__(self, embed_dim, num_heads):
        super().__init__()

        self.num_heads = num_heads
        self.d = embed_dim // num_heads

        self.W_q = nn.Linear(embed_dim, embed_dim, bias=False)
        self.W_k = nn.Linear(embed_dim, embed_dim, bias=False)
        self.W_v = nn.Linear(embed_dim, embed_dim, bias=False)
        self.W_o = nn.Linear(embed_dim, embed_dim, bias=False)

    def forward(self, X, enc_out):

        B, N, _ = X.shape
        _, S, _ = enc_out.shape

        Q = self.W_q(X).view(B, N, self.num_heads, self.d).transpose(1, 2)
        K = self.W_k(enc_out).view(B, S, self.num_heads, self.d).transpose(1, 2)
        V = self.W_v(enc_out).view(B, S, self.num_heads, self.d).transpose(1, 2)

        _, out = scaled_dot_product_attention(Q, K, V)

        out = out.transpose(1, 2).contiguous().view(B, N, self.num_heads * self.d)
        return self.W_o(out)
```

The full decoder block therefore consists of three attention layers (masked self-attention, cross-attention, and feed-forward) interleaved with residual connections and layer normalization:

```
class DecoderBlock(nn.Module):

    def __init__(self, embed_dim, num_heads, ffn_dim, dropout=0.0):
        super().__init__()

        # Masked self-attention over the target sequence
        self.masked_attention = MaskedMultiHeadAttention(embed_dim, num_heads)

        # Cross-attention: reads from the encoder output
        self.cross_attention = CrossAttention(embed_dim, num_heads)

        # Feed-forward sublayer
```

```
self.linear1 = nn.Linear(embed_dim, ffn_dim)
self.linear2 = nn.Linear(ffn_dim, embed_dim)

# Layer normalisation – one per sublayer
self.norm1 = nn.LayerNorm(embed_dim)
self.norm2 = nn.LayerNorm(embed_dim)
self.norm3 = nn.LayerNorm(embed_dim)

self.dropout = nn.Dropout(dropout)

def forward(self, X, enc_out):

    # Masked self-attention sublayer
    X = self.norm1(X + self.dropout(self.masked_attention(X)))

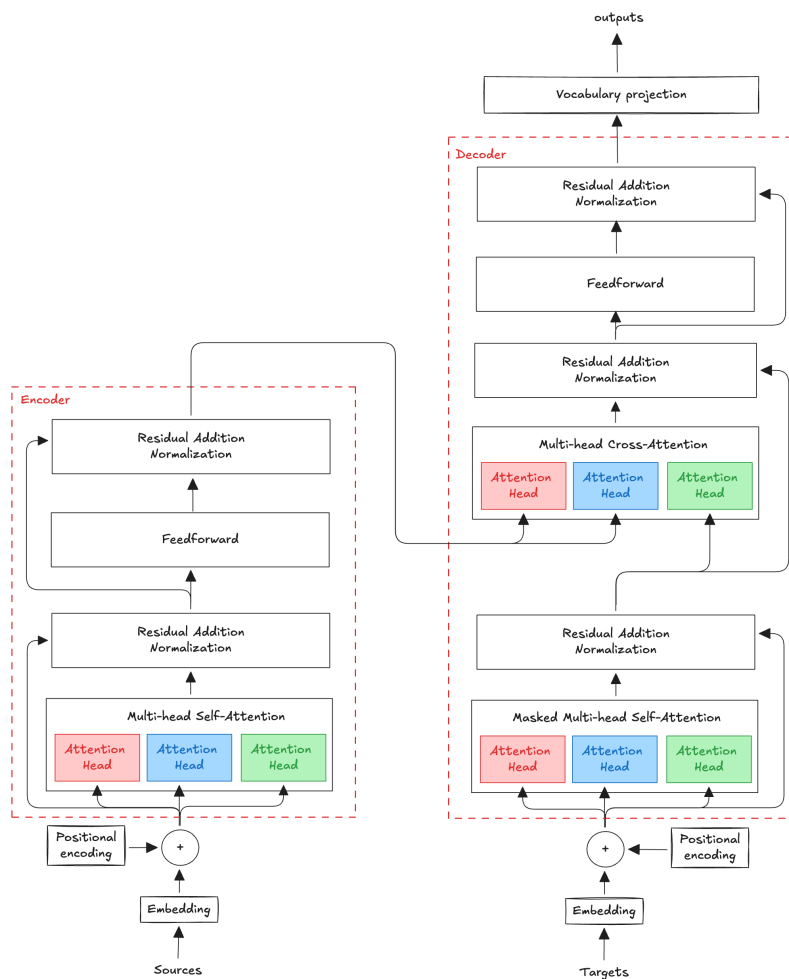
    # Cross-attention sublayer
    X = self.norm2(X + self.dropout(self.cross_attention(X, enc_out)))

    # Feed-forward sublayer
    ffn_out = self.linear2(F.relu(self.linear1(X)))
    X = self.norm3(X + self.dropout(ffn_out))

    return X
```

1.7.6 The full transformer

All the pieces are now in place. We can assemble a full Transformer by stacking encoder and decoder blocks, wiring in the shared embeddings and positional encoding, and adding a final linear projection from the embedding space to the target vocabulary. The model separates encoding and decoding into two explicit methods so that during inference the source can be encoded once and the decoder can generate autoregressively token by token.



```
class Transformer(nn.Module):

    def __init__(self, embed_dim, num_heads, ffn_dim, num_layers, dropout=0.0):
        super().__init__()

        # Source and target embeddings
        self.src_embedding = Embedding(eng_vocab_size, embed_dim)
        self.tgt_embedding = Embedding(ita_vocab_size, embed_dim)

        # Shared positional encoding
        self.pos_encoding = PositionalEncoding(embed_dim)

        # Stack of encoder blocks
        self.encoder = nn.ModuleList([
            EncoderBlock(embed_dim, num_heads, ffn_dim, dropout)
            for _ in range(num_layers)
        ])
```

```
    ])  
  
    # Stack of decoder blocks  
    self.decoder = nn.ModuleList([  
        DecoderBlock(embed_dim, num_heads, ffn_dim, dropout)  
        for _ in range(num_layers)  
    ])  
  
    # Final projection from embedding space to target vocabulary  
    self.output_projection = nn.Linear(embed_dim, ita_vocab_size)  
  
    def forward(self, src, tgt):  
  
        # Encode the source sequence through the stack of encoder blocks to get the  
        ↪ encoder output  
        enc_out = self.encode(src)  
  
        # Decode the target sequence through the stack of decoder blocks, attending to  
        ↪ the encoder output  
        dec_out = self.decode(tgt, enc_out)  
  
        # Project the decoder output onto the target vocabulary to get the logits for  
        ↪ the next token prediction  
        return self.output_projection(dec_out)  
  
    # Internal method to encode the source sequence through the stack of encoder blocks  
    def encode(self, src):  
        X = self.pos_encoding(self.src_embedding(src))  
        for block in self.encoder:  
            X = block(X)  
        return X  
  
    # Internal method to decode the target sequence through the stack of decoder blocks  
    def decode(self, tgt, enc_out):  
        X = self.pos_encoding(self.tgt_embedding(tgt))  
        for block in self.decoder:  
            X = block(X, enc_out)  
        return X
```

1.7.7 Performance

We instantiate the Transformer with the same embedding dimension and number of layers used in the RNN baselines, so that any difference in performance can be attributed to the architecture rather than to scale.

```
# Create an instance of the Transformer model
transformer_model = Transformer(embed_dim = 256, num_heads = 8, ffn_dim = 1024,
    ↪ num_layers = 2, dropout = 0.1).to(device)

# Create an optimizer for the Transformer model parameters
transformer_optimizer = torch.optim.Adam(transformer_model.parameters(), lr=1e-3)
```

Also in this case we check the number of trainable parameters:

```
# Compute the total number of trainable parameters in the Transformer model
total_params = sum(p.numel() for p in transformer_model.parameters() if
    ↪ p.requires_grad)

# Print the total number of trainable parameters in the Transformer model
print(f"Transformer parameters: {total_params:,}")
```

Transformer parameters: 17,681,858

We train the attention model for some epochs using the same procedure used before, caching the weights and loss curves to disk so that subsequent runs skip training and reload the checkpoint directly:

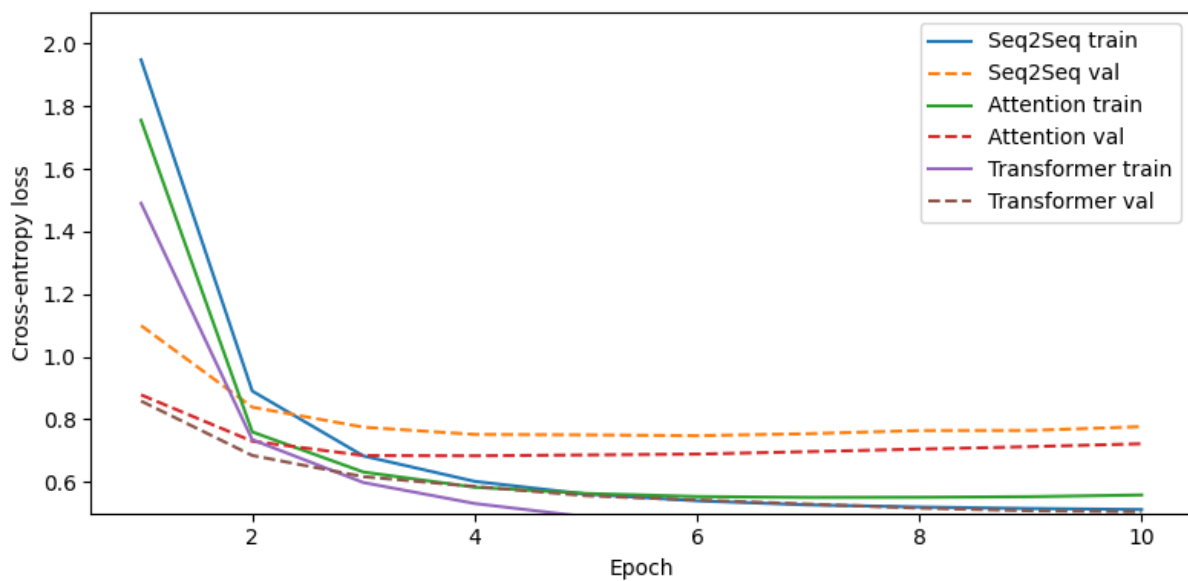
```
# Define the path to the model checkpoint file
model_path = "./data/transformer_model.pt"

# Train the Transformer model and obtain the training and validation losses for each
    ↪ epoch
transformer_train_losses, transformer_val_losses = train_model(transformer_model,
    ↪ transformer_optimizer, model_path)
```

Epoch 10 | train loss 0.3839 | val loss 0.5048
Model saved to ./data/transformer_model.pt

We plot the training and validation loss curves for all the models to visually compare the training dynamics:

```
plt.figure(figsize=(8, 4))
plt.plot(range(1, num_epochs + 1), seq2seq_train_losses, label='Seq2Seq train')
plt.plot(range(1, num_epochs + 1), seq2seq_val_losses, label='Seq2Seq val',
        linestyle='--')
plt.plot(range(1, num_epochs + 1), attention_train_losses, label='Attention train')
plt.plot(range(1, num_epochs + 1), attention_val_losses, label='Attention val',
        linestyle='--')
plt.plot(range(1, num_epochs + 1), transformer_train_losses, label='Transformer
        train')
plt.plot(range(1, num_epochs + 1), transformer_val_losses, label='Transformer val',
        linestyle='--')
plt.xlabel('Epoch')
plt.ylabel('Cross-entropy loss')
plt.ylim(0.5, 2.1)
plt.legend()
plt.tight_layout()
plt.show()
```



Three things stand out immediately. First, **convergence speed**. The Transformer drops steeply in the first few epochs. The attention model follows a similar trajectory. The plain Seq2Seq takes much longer to descend. Second, **final training loss**. All three models converge to roughly the same training loss, which tells us the models have similar capacity and the task is equally learnable by all three given enough epochs. Third, and most telling, the **train-validation gap**. The

Seq2Seq shows the worst generalisation: the validation curve plateaus is high and even drifts slightly upward, while training keeps improving. The attention model closes that gap noticeably. The Transformer is the striking result: the validation curve sits almost on top of the training curve throughout, finishing with essentially no gap at all. On this dataset, the Transformer is not overfitting while the RNN models clearly are. This last point is worth emphasising: the Transformer achieves better validation loss not because it has more parameters or trains longer, but because **its representations generalise better**. The causal mask and cross-attention give it a more structured inductive bias for sequence-to-sequence tasks than a GRU hidden state.

```
def transformer_predict(source, max_len=20):

    # Set the model to evaluation mode
    transformer_model.eval()

    # Disable gradient computation since we are only performing inference
    with torch.no_grad():

        # Get the indices for the beginning-of-sequence and end-of-sequence
        # tokens from the Italian vocabulary
        bos_idx = ita_to_idx['<bos>']
        eos_idx = ita_to_idx['<eos>']

        # Encode the source sequence once
        source = ['<bos>'] + source + ['<eos>']
        src_ids = torch.tensor(encode(source, eng_to_idx,
↪ length=20)).unsqueeze(0).to(device)
        enc_out = transformer_model.encode(src_ids)

        # Decode autoregressively starting from <bos>
        tgt = [bos_idx]
        for _ in range(max_len):
            tgt_tensor = torch.tensor(tgt).unsqueeze(0).to(device)
            dec_out = transformer_model.decode(tgt_tensor, enc_out)
            next_token = transformer_model.output_projection(dec_out[:, -1,
↪ :]).argmax(dim=-1).item()
            if next_token == eos_idx:
                break
            tgt.append(next_token)

        return [idx_to_ita.get(i, '<unk>') for i in tgt[1:]]
```

We evaluate the model on the validation set using the same BLEU metric computed earlier, so

we can directly compare translation quality respect to the previous models:

```
# Evaluate the BLEU score of the attention-based model on the test set
transformer_bleu_overall, transformer_bleu_by_length =
    evaluate_bleu(transformer_model, transformer_predict)

# Print the overall BLEU score for all models to compare their performance
print(f"Seq2Seq Overall BLEU : {seq2seq_bleu_overall:.4f}")
print(f"Attention Overall BLEU: {attention_bleu_overall:.4f}")
print(f"Transformer Overall BLEU: {transformer_bleu_overall:.4f}")
```

Seq2Seq Overall BLEU : 0.4960

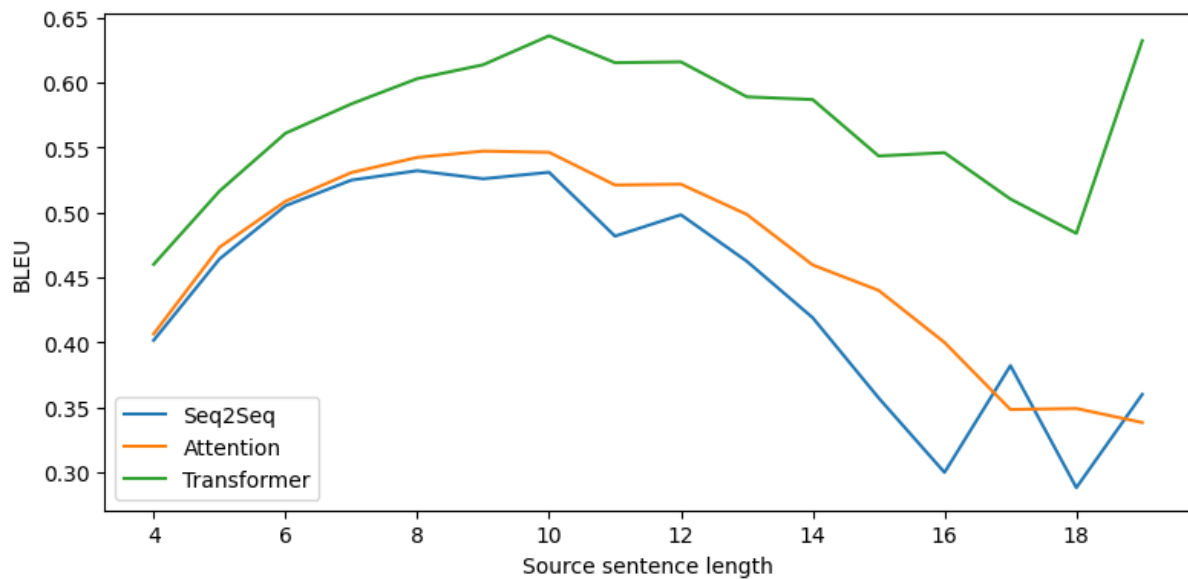
Attention Overall BLEU: 0.5072

Transformer Overall BLEU: 0.5667

The Transformer achieves the highest overall BLEU score, confirming that the improvement seen in the loss curves translates into better translation quality: each step in the architectural progression, from fixed context vector to dynamic attention to full self-attention, produces a measurable gain, with the Transformer scoring well above the plain Seq2Seq baseline. To understand where the improvement comes from, we break down the BLEU score by source sentence length:

```
# Extract the BLEU scores for the selected lengths
transformer_bleus = [transformer_bleu_by_length[l] for l in lengths]

# Create a line plot of BLEU scores by source sentence length
plt.figure(figsize=(8, 4))
plt.plot(lengths, seq2seq_bleus, label='Seq2Seq')
plt.plot(lengths, attention_bleus, label='Attention')
plt.plot(lengths, transformer_bleus, label='Transformer')
plt.xlabel('Source sentence length')
plt.ylabel('BLEU')
plt.legend()
plt.tight_layout()
plt.show()
```



The length breakdown reveals the architectural differences far more clearly than the overall BLEU score alone. On short sentences all three models are competitive, with the gap between them narrow. As sentences grow longer the curves diverge sharply: the plain Seq2Seq degrades the most, a direct consequence of the fixed context vector bottleneck struggling to compress longer inputs. The attention model resists the degradation better but still drops significantly, confirming that dynamic alignment helps but the recurrent backbone still limits capacity. The Transformer holds its score remarkably well across the entire range, and even then the drop is much shallower than the other two. This is exactly the behaviour the architecture is designed to produce: because every token in the encoder output remains directly accessible to the decoder at every step via cross-attention, longer sequences do not impose any additional compression cost, and translation quality degrades much more gracefully with length.

Finally, we can inspect a few hand-picked sentences to get a qualitative feel for where the Transformer helps most:

```
# For each previously defined example sentence, compare translations using all three
↳ models:
for src in examples:
    base      = seq_2_seq_predict(src)
    att, _    = attention_predict(src)
    trans     = transformer_predict(src)
    print(f" EN:           {' '.join(src)}")
    print(f" Seq2Seq:       {' '.join(base)}")
    print(f" Attention:      {' '.join(att)}")
    print(f" Transformer:    {' '.join(trans)}")
```

```
print()
```

EN: i love you .

Seq2Seq: io vi amo .

Attention: ti amo .

Transformer: vi amo .

EN: the cat is on the table .

Seq2Seq: il gatto è sdraiato sul tavolo .

Attention: il gatto è sul tavolo .

Transformer: il gatto è sul tavolo .

EN: where is the train station ?

Seq2Seq: dov ' è la stazione ferroviaria ?

Attention: dov ' è la stazione dei treni ?

Transformer: dov ' è la stazione ferroviaria ?

EN: good morning !

Seq2Seq: buongiorno !

Attention: buongiorno !

Transformer: buongiorno !

EN: she reads a book every day .

Seq2Seq: lei legge un libro ogni giorno .

Attention: legge un libro ogni giorno .

Transformer: legge un libro ogni giorno .

EN: we are happy .

Seq2Seq: siamo felici .

Attention: siamo felici .

Transformer: siamo felici .

EN: what time is it ?

Seq2Seq: che ora è ?

Attention: che ore è ?

Transformer: che ora è ?

EN: can you help me ?
Seq2Seq: potete aiutarmi ?
Attention: puoi aiutarmi ?
Transformer: potete aiutarmi ?

EN: this is a test .
Seq2Seq: questo è un esame .
Attention: questo è un test .
Transformer: questo è un esame .

The qualitative examples reinforce the quantitative results. All three models handle simple, frequent sentences equally well, but differences emerge on harder cases: the Transformer drops the redundant subject pronoun in "ti amo" and "legge un libro ogni giorno", which is grammatically correct in Italian, produces the idiomatic "buongiorno" rather than the literal "buona mattina", gives the natural plural "che ore sono" instead of the malformed "che ora è ora", and is the only model to preserve the English loanword "test" rather than substituting the unrelated "esame". These are small differences on a limited dataset, but they point in a consistent direction, the Transformer has learned slightly more idiomatic and contextually accurate translations than its recurrent counterparts.